

逆引き R 関数

藤野秀則,

2025-02-07

目次

1	ライブラリの読み込み	4
2	データ読み込み	5
2.1	csv ファイルを読み込みこむ.	5
2.2	データフレームオブジェクトから欠損値を含む行を一括で削除する.	5
3	データの確認	5
3.1	オブジェクトの型を確認する	5
3.2	オブジェクトの構造を確認する. (class() よりも詳細な情報を得る)	5
3.3	オブジェクトの要約情報をを表示する.	6
3.4	オブジェクトの先頭から指定した行数分を表示する.	6
3.5	オブジェクトの要約統計量をを表示する.	6
3.6	データフレームに含まれる変数名 (列名) を表示する.	6
3.7	出力	6
4	基本的な処理	7
4.1	算術	7
4.1.1	絶対値を求める.	7
4.1.2	数値の小数部を四捨五入する.	7
4.1.3	数値の小数部を切り捨てるする.	7
4.1.4	数値の小数部を切り上げる.	7
4.1.5	平方根を求める	8
4.1.6	合計を求める	8
4.2	データの操作	8
4.2.1	ベクトルの結合	8
4.2.2	データフレームの作成	8
4.2.3	データフレームを列方向に結合する	8
4.2.4	データフレームオブジェクトを行方向に結合する	9
4.2.5	連続した数値ベクトルを生成する	9

4.2.6	ベクトルデータから指定した要素を抽出する	9
4.2.7	データフレームオブジェクトから指定した行, 列を抽出する.	9
4.2.8	データフレームオブジェクトから指定した列を抽出する.	9
4.2.9	ベクトルに含まれる要素数を求める	10
4.2.10	データフレームの行数 (サンプル数) を求める.	10
4.2.11	データフレームの列数 (変数数) を求める.	10
4.3	型の変換	10
4.3.1	オブジェクトを要因型に変換する.	10
4.3.2	オブジェクトを数値型に変換する.	10
4.3.3	オブジェクトを文字列型に変換する.	10
4.3.4	行列型オブジェクトをデータフレーム型に変換する.	11
5	抽出・加工・置換・ロング-ワイドの変換	11
5.1	データの抽出	11
5.1.1	条件に合致するデータの抽出	11
5.1.2	列の抽出	12
5.2	データフレームへの変数の追加	12
5.2.1	1 つの変数を追加する.	12
5.2.2	複数の変数をまとめて追加する	12
5.3	変数に含まれているデータの置換	12
5.3.1	その 1 <code>if_else()</code>	13
5.3.2	その 2 <code>recode()</code>	13
5.3.3	その 3 <code>case_when()</code>	14
5.4	ワイド型データからロング型データへの変換	14
5.5	ロング型データからワイド型データへの変換	15
6	データの記述統計量を得る	15
6.1	1 変数の単純な記述統計量を得る	15
6.1.1	数値ベクトルの平均値を求める.	15
6.1.2	数値ベクトルの標準偏差を求める.	15
6.1.3	数値ベクトルの分散を求める.	15
6.1.4	数値ベクトルのの最大値を求める.	16
6.1.5	数値ベクトルのの最小値を求める.	16
6.1.6	数値ベクトルのの中央値を求める.	16
6.1.7	数値ベクトルのの分位数を求める.	16
6.1.8	度数分布を求める.	17
6.1.9	1 変数の標準誤差を得る	17
6.1.10	1 変数の平均値の信頼区間を得る	17
6.2	グループごとの記述統計量を得る	17
6.2.1	1 つの変数に対して	17
6.2.2	複数の変数に対して	18

6.2.3	1つの変数に対して、複数のグループ分け基準を指定して	18
6.2.4	複数の変数に対して、複数のグループ分け基準を指定して	18
6.2.5	複数の集計関数を同時に適用	18
7	比較をしたい	19
7.1	2群で平均値を比較したい	19
7.1.1	データに対応関係がない場合	19
7.1.2	データに対応関係がある場合	19
7.2	2群で分散を比較したい	20
7.3	2群で比率を比較したい	20
7.4	2つの要因の独立性を検定したい	20
7.4.1	カイ二乗検定	20
7.4.2	Fisher の正確確率検定	21
7.5	3群以上での比較をしたい	21
8	相関を調べたい	21
8.1	相関係数を求めたい	21
8.2	相関係数の検定をしたい	22
8.3	相関の差の検定をしたい	22
8.3.1	独立サンプルにおける相関の差の検定	22
8.3.2	同一サンプルにおける1つの変数Jが共通している相関の差の検定	22
8.3.3	同一サンプルにおける互いに異なる変数間の相関の差の検定	23
9	連続変数間での予測式を作りたい・因果関係を調べたい (回帰分析)	23
9.1	単純な重回帰分析	23
9.2	標準化重回帰分析	24
9.3	交互作用項を含む重回帰分析	25
9.4	一部の変数をカテゴリカル変数として扱いたい	25
9.5	ステップワイズ法による変数選択	25
10	離散変数と連続変数の間の因果関係を調べたい (分散分析)	26
10.1	1 要因分散分析	26
10.1.1	対応なしデータ/被験者間計画	26
10.1.2	対応ありデータ/被験者内計画	26
10.1.2.1	その1	26
10.1.2.2	その2	26
10.2	2 要因分散分析	27
10.2.1	対応なしデータ/被験者間計画	27
10.2.1.1	その1	27
10.2.1.2	その2	28
10.2.2	対応ありデータ/被験者内計画	28
10.2.2.1	その1	28

10.2.2.2	その 2	29
10.2.3	一方の変数だけ対応ありデータ/混合計画	29
10.2.3.1	その 1	29
10.2.3.2	その 2	29
11	変数を集約したい	30
11.1	主成分分析	30
11.1.1	主成分負荷量を得たい	30
11.1.2	主成分得点を得たい	30
11.2	因子分析	31
11.2.1	因子数を模索したい	31
11.2.1.1	スクリープロットを得たい	31
11.2.1.2	vss 法を実施したい	31
11.2.2	探索的因子分析を実施したい	31
11.2.3	因子分析の結果を出力させたい	32
11.2.4	因子分析の結果を図示したい	32
11.2.5	因子得点を得たい	32
12	サンプルをグルーピングしたい	33
12.1	階層的クラスタリング法	33
12.2	k-means 法	33
13	ggplot2 を用いたデータの可視化	34
13.1	キャンバスの作成	34
13.2	散布図を描く	35
13.3	ヒストグラム（度数分布表）を描く	37
13.4	箱ひげ図を描く	40
13.5	グループごとの平均の棒グラフとエラーバーを描く	43
13.5.1	データそのものを与える場合	43
13.5.1.1	エラーバーの追記	44
13.5.2	平均や標準偏差を算出して与える場合	45
13.5.2.1	エラーバーの追記	46
13.6	回帰直線と信頼区間を描く	47
13.7	任意の直線を描く	49

1 ライブラリの読み込み

- 例文: `library(tidyverse)`
- 引数: 読み込むライブラリ名
- 返回值: 特になし
- 留意点: ライブラリを読み込む前に、予めライブラリを自身の Posit Cloud や RStudio にインストー

ルする必要がある。一度インストールしておけば、その後は `library()` で読み込むだけでよい。また、一度 `library()` で読み込んだライブラリはその実行環境上、あるいは、その Rmarkdown ファイル上ではずっと有効になる。

2 データ読み込み

2.1 csv ファイルを読み込みこむ。

- 例文: `data <- read.csv("./myfolder/data.csv", header=TRUE, encoding="UTF-8")`
- 引数:
 - 第 1 引数: ファイルのパス名。Posit Cloud 上でフォルダを作って、フォルダの中に CSV ファイルがある場合には `"/フォルダ名/ファイル名.csv"` と指定する。
 - 第 2 引数 `header=TRUE`: ヘッダーがある場合に指定
 - 第 3 引数 `encoding="UTF-8"`: 文字コードを指定。特に UTF-8 で文字化けしてうまく読み込めない場合には `"SJIS"` を試す (Excel で作成した CSV ファイルは SJIS にする必要がある場合が多い)。
- 返回值: 読み込んだデータを格納したデータフレーム

2.2 データフレームオブジェクトから欠損値を含む行を一括で削除する。

- 例文: `data <- na.omit(data)`
- 引数: 欠損値を含むデータフレームオブジェクト
- 返回值: 欠損値を含まないデータフレームオブジェクト

3 データの確認

3.1 オブジェクトの型を確認する

- 例文: `class(data)`
- 引数: 確認したいオブジェクト
- 返回值: オブジェクトの型

3.2 オブジェクトの構造を確認する。(class() よりも詳細な情報を得る)

- 例文: `str(data)`
- 引数: 構造確認したいオブジェクト
- 返回值: オブジェクトに含まれる変数名、変数の型、サンプル数、欠損値の有無などが表示される。

3.3 オブジェクトの要約情報を表示する。

- 例文: `summary(data)`
- 引数: データの概要を表示したいデータフレーム、もしくはベクトル
- 返り値: 数値の場合には、最小値、第1四分位数、中央値、平均値、第3四分位数、最大値が表示される。要因型の場合には、水準名と要素数が表示される。文字列型の場合にはサンプル数と `character` 型であることをが表示される。
- 留意点: 与えるオブジェクトのクラスによって、表示される情報が異なっている。上記はあくまでデータフレームやベクトルを与えた場合の出力。

3.4 オブジェクトの先頭から指定した行数分を表示する。

- 例文: `head(data, 5)`
- 引数:
 - 第1引数: 表示したいデータフレーム、もしくはベクトル
 - 第2引数: 表示したい行数。省略した場合には6行分が表示される。
- 返り値: 指定した行数分のデータフレームオブジェクト。そのまま実行した場合、6行分が表示される。

3.5 オブジェクトの要約統計量を表示する。

- 例文: `describe(data)`
- 必要ライブラリ: `psych`
- `summary()` と異なり、平均値、標準偏差などが表示される。
- 引数: 要約統計量を表示したいデータフレーム、もしくはベクトル

3.6 データフレームに含まれる変数名（列名）を表示する。

- 例文: `colnames(data)`
- 引数: 列名を表示したいデータフレーム
- 返り値: データフレームに含まれる変数名のベクトル

3.7 出力

- 例文: `print(data)`
- 引数: 出力したいオブジェクト

- 留意点: 単純な数値や、ベクトル、データフレームなどは、`print` を付けなくても、オブジェクト名をそのまま実行するだけで出力されるが、オブジェクトによっては `print()` での出力をした場合と、そのまま実行した場合とで出力結果が異なる場合がある。その場合は `print()` での出力が基本となる。

4 基本的な処理

4.1 算術

4.1.1 絶対値を求める。

- 例文: `abs(data)`
- 引数: 絶対値を求めたい数値を含むオブジェクト
- 返り値: 絶対値

4.1.2 数値の小数部を四捨五入する。

- 例文: `round(data, 2)`
- 引数:
 - 第 1 引数: 数値を含むオブジェクト
 - 第 2 引数: 有効にする小数点以下の桁数。この桁数までが表記される
- 返り値: 四捨五入された数値
- 留意点: あくまで少数以下の指定の桁数までを四捨五入する、というものである。したがって、整数位での四捨五入をするためには、一旦、全体を $1/100$ や $1/1000$ をして、小数位にまで落としてから `round()` を適用し、後で 100 や 1000 でかけるといった処理が必要になる。

4.1.3 数値の小数部を切り捨てるする。

- 例文: `floor(data)`
- 引数: 少数以下を切り捨てたい数値を含むオブジェクト
- 返り値: 切り捨てられた整数値
- 留意点: 小数以下は全て切り捨てられるので、例えば小数第 1 位まで残したい場合には、全体を 10 倍してから `floor()` を掛け、後で 10 で割るといった処理が必要になる。

4.1.4 数値の小数部を切り上げる。

- 例文: `ceiling(data)`
- 引数: 少数以下を切り上げたい数値を含むオブジェクト
- 返り値: 切り上げられた整数値

- 留意点: 小数以下は全て切り上げられるので、例えば小数第 1 位まで残したい場合には、全体を 10 倍してから `ceiling()` を掛け、後で 10 で割るといった処理が必要になる。

4.1.5 平方根を求める

- 例文: `qrt(data)`
- 引数: 平方根を求めたい数値を含むオブジェクト
- 返回值: 平方根

4.1.6 合計を求める

- 例文: `sum(data)`
- 引数: 合計値を求めたい数値ベクトルのオブジェクト
- 返回值: 合計値

4.2 データの操作

4.2.1 ベクトルの結合

- 例文: `vec <- c(data1, data2)`
- 引数: 結したいベクトルを順に並べる。幾つでも並べることができる。
- 返回值: 結合されたベクトル。
- 留意点: もし数値ベクトルと文字列ベクトルが混在している場合には、数値ベクトルが文字列として扱われ、文字列ベクトルが出力される

4.2.2 データフレームの作成

- 例文: `df <- data.frame(d1=data1, d2=data2, d3=data3)`
- 引数: データフレームにふくめるベクトルデータを順に並べる。幾つでも並べることができる。
変数名 = ベクトルデータ とするとデータフレーム上での変数名を設定できる。変数名の設定を省略した場合には、ベクトル名が変数名となる。
- 返回值: データフレームオブジェクト
- 留意点: 与える各ベクトルはデータの長さが一致している必要がある。

4.2.3 データフレームを列方向に結合する

- 例文: `df<-cbind(data1, data2)`
- 引数: 結合したいデータフレームオブジェクトを順に並べる。幾つでも並べることができる。
- 返回值: 結合されたデータフレームオブジェクト

- 留意点: 与えるデータフレームオブジェクトの行数は一致している必要がある。

4.2.4 データフレームオブジェクトを行方向に結合する

- 例文: `df <- rbind(data1, data2)`
- 引数: 結合したいデータフレームオブジェクトを順に並べる。幾つでも並べることができる。
- 返り値: 結合されたデータフレームオブジェクト
- 留意点: 与えるデータフレームオブジェクトの列数は一致している必要がある。

4.2.5 連続した数値ベクトルを生成する

- 例文: `vec<-1:10`
- 引数: 連続した数値の最初の値と最後の値で `:` を挟む。
- 返り値: 指定した範囲の連続した数値ベクトル

4.2.6 ベクトルデータから指定した要素を抽出する

- 例文: `data[3]`, `data[1:10]`, `data[c(1,3,5)]`, `data[-1]`, `data[c(-1,-4)]`
- 引数: 抽出したい要素のインデックス番号。複数のものを取り出したい場合には `c(1,3,5:10)` のようにベクトルで指定。
- 返り値: 抽出した要素だけのベクトルデータ。
- 留意点: 負の値を指定するとその列が除外されたものが返される。負の値と正の値を混在させることはできない。

4.2.7 データフレームオブジェクトから指定した行、列を抽出する。

- 例文:
`data[3,5]`, `data[c(1,3,5:10), c(1,3,5:10)]`, `data[c(1,3,5:10), ]`,
`data[, c(1,3,5:10)]`, `data[c(1,3,5:10)]`
- 引数: 抽出したい箇所の行のインデックス番号と列のインデックス番号を `,` で区切って与える。行番号, 列番号とも記載方法はベクトルデータの抽出と同様。いずれかを省略した場合, すべての行, もしくは列が抽出される。また, `data[c("変数名 1", "変数名 2", "変数名 3")]` のように各列の変数名を指定した場合, すべての行が抽出される。
- 返り値: 抽出したデータフレームオブジェクト。

4.2.8 データフレームオブジェクトから指定した列を抽出する。

- 例文: `data$var1`
- 引数: 抽出したい列の変数名。

- 戻り値: 抽出した列のベクトルデータ。
- 留意点: `data["var1"]` と `data$var1` は似たような効果を持つが、前者はあくまで 1 変数で構成されたデータフレームを返すのに対して、後者はベクトルを返す。

4.2.9 ベクトルに含まれる要素数を求める

- 例文: `length(data)`
- 引数: 要素数を求めたいベクトルデータ
- 戻り値: ベクトルに含まれる要素数

4.2.10 データフレームの行数（サンプル数）を求める。

- 例文: `nrow(data)`
- 引数: 行数を求めたいデータフレームオブジェクト
- 戻り値: データフレームオブジェクトの行数（サンプル数）

4.2.11 データフレームの列数（変数数）を求める。

- 例文: `ncol(data)`
- 引数: 列数を求めたいデータフレームオブジェクト
- 戻り値: データフレームオブジェクトの列数（変数数）

4.3 型の変換

4.3.1 オブジェクトを要因型に変換する。

- 例文: `as.factor(data$var1)`
- 引数: 要因型に変換したいオブジェクト
- 戻り値: 要因型に変換されたオブジェクト

4.3.2 オブジェクトを数値型に変換する。

- 例文: `as.numeric(data$var1)`
- 引数: 数値型に変換したいオブジェクト
- 戻り値: 数値型に変換されたオブジェクト

4.3.3 オブジェクトを文字列型に変換する。

- 例文: `as.character(data$var1)`

- 引数: 文字列型に変換したいオブジェクト
- 返り値: 文字列型に変換されたオブジェクト

4.3.4 行列型オブジェクトをデータフレーム型に変換する.

- 例文: `as.data.frame(data)`
- 引数: データフレーム型に変換したい行列型オブジェクト.
- 返り値: データフレーム型に変換されたオブジェクト.
- 留意点: `scale()` 関数の出力や, `prcomp()` で得られる主成分得点のデータなど, 一部の関数の出力は行列型となっており, そのまま利用した場合に, うまく処理がされないケースがある. そういう場合は, `as.data.frame()` でデータフレーム型に変換してから利用する.

5 抽出・加工・置換・ロング-ワイドの変換

以下では, いずれも実行に当たっては当たっては, `library(tidyverse)` を読み込んでおくことを前提とする.

5.1 データの抽出

5.1.1 条件に合致するデータの抽出

- 例文: `filter(data, var1>10 & var2=="A")`
- 引数:
 - 第 1 引数: 抽出元のデータフレームオブジェクト
 - 第 2 引数: 抽出条件を指定する. 条件指定の主な書き方は以下の通り. 複数の条件を指定する場合, `&` を使って条件をつなげると**すべての条件を満たすデータ**が, `|` を使って条件をつなげると**いずれかの条件を満たすデータ**が抽出される.
 - * `var1>10`: 指定した値より**大きい**データを抽出
 - * `var1>=10`: 指定した値**以上**のデータを抽出
 - * `var1<10`: 指定した値より**小さい**データを抽出
 - * `var1<=10`: 指定した値**以下**のデータを抽出
 - * `var1==10`: 指定した値と等しいデータを抽出. 文字列型や要因型でも利用可能.
 - * `var1!=10`: 指定した値と異なるデータを抽出. 文字列型や要因型でも利用可能.
 - * `var1 %in% c(1,2,3)`: 指定した値のいずれかに合致するデータを抽出. 要因型でも利用可能.
- 返り値: 指定した条件で抽出されたデータフレームオブジェクト

5.1.2 列の抽出

- 例文: `select(data, var1, var2)`, `select(data, -var4, -var6)`
- 引数:
 - 第1引数: 抽出したいデータフレームオブジェクト
 - 第2引数: 抽出したい列を順に並べていく. `-`を使って列を指定すると, その(それらの)列以外の全ての列が抽出される.
- 返回值: 指定した列で抽出されたデータフレームオブジェクト
- 留意点: `[]` を使って列を抽出するものと同じ効果を持つ. パイプライン処理をするときには `select()` を使う.

5.2 データフレームへの変数の追加

データフレームに含まれる変数やその他のベクトルを使って新たな変数を生成し, 元のデータフレームに追加する.

5.2.1 1つの変数を追加する.

- 例文: `data$var3 <- data$var1+data$var2+ex_val`
- 留意点: 単純にベクトルの演算を行ったうえで, 元のデータフレームに `$` で新たな変数名を指定して, 格納すればよい.

5.2.2 複数の変数をまとめて追加する

- 例文: `mutate(data, var3=var1+var2, var4=var1-var2, var5=var1*ex_val)`
- 引数:
 - 第1引数: 追加したいデータフレームオブジェクト
 - 第2引数以降: 追加したい列名とその計算式を順に並べていく. 幾つでも並べることができる. データフレームに含まれる変数は変数名を指定するだけでよい. またデータフレーム外の変数を用いることもできる.
- 返回值: 新たな列が追加されたデータフレームオブジェクト
- 留意点: データフレームに含まれる変数に何らかの演算処理を行って新たなデータを作り, 元のデータオブジェクトに格納する.

5.3 変数に含まれているデータの置換

`if_else()`, `recode()`, `case_when()` の3つの関数がある.

`if_else()` と `case_when()` は数値、文字列いずれの置換にも対応。一方、`recode()` は文字列からの置換のみに対応。

また、`if_else()` は複数の置換条件を置く際には入れ子にしていく必要があるが、`recode()` と `case_when()` は複数の置換条件を並べて書くだけでよい。

いずれも `tidyverse` パッケージ（正確には `dplyr` パッケージ）を用いる。

5.3.1 その1 `if_else()`

- 例文 1: `data$new <- if_else(data$var1>3, "A", "B", missing="errorr")` (数字から文字列)
- 例文 2: `data$new <- if_else(data$var1>3, 1, 0, missing=NA)` (数字から数字)
- 例文 3: `data$new <- if_else(data$var1=="X", "A", "B")` (文字列から文字列)
- 例文 4: `data$new <- if_else(data$var1=="X", 1, 0)` (文字列から数字)
- 例文 5: `data$new <- if_else(data$var1>3, "A", if_else(data$var1>1,"B","c"))` (入れ子)
- 例文 6: `data$new <- if_else(data$var1>3 & data$var2=="Y", "A", "B")` (複数の変数を条件にする)
- 引数:
 - 第 1 引数: 置換条件を指定。条件の書き方は先に紹介したものと同一。
 - 第 2 引数: 条件が真の場合に置換される値を指定。
 - 第 3 引数: 条件が偽の場合に置換される値を指定。
 - 第 4 引数 `missing="error"` : `NA` (欠損値) がある場合に、`NA` を置換する値を指定。省略した場合、`NA` はそのまま `NA` となる。
- 返り値: 置換後の変数
- 留意点: 文字列、数値のいずれにも対応 (例文 1–4)。第 2 引数や第 3 引数に `if_else()` を入れ子にして利用することもできる (例文 5)。また、条件の中では異なる変数を組み合わせた条件を作ることできる (例文 6)。先にで紹介した `mutate()` と組み合わせて使うことが多い。`mutate()` の中で用いる場合には、`data$` をつける必要はない。

5.3.2 その2 `recode()`

- 例文 1: `data$new <- recode(data$var1, "x1"="A", "x2"="B", "x3"="C", .default="error")` (文字列から文字列)
- 例文 2: `data$new <- recode(data$var1, "x1"=1, "x2"=2, "x3"=3, .default=0)` (文字列から数字)
- 引数:
 - 第 1 引数: 置換対象の変数
 - 第 2 引数以降: 置換対象の値と置換後の値を `=` で結んで指定。
 - 最終引数 `.default=` (省略可): すべての条件に合致しない場合の置換値を指定する。省略した場合、すべての条件に合致しないものは、例文 1 の場合には元の値が保持され、例文 2 の場合に

は NA に置換される。

- 返り値: 置換後の変数
- 留意点: 文字列からの置換のみに対応。 `if_else()` のように入れ子にしくなくても、複数の置換条件を指定することができる。 `mutate()` の中で用いる場合には、 `data$` をつける必要はない。

5.3.3 その3 case_when()

- 例文:

```
data$new<-case_when(data$var1>3 ~ "A",  
                    data$var1<=3 & data$var2 == "Y" ~ "B",  
                    TRUE ~ "C")
```

データフレームに含まれるデータから条件に応じて値を置換した変数を作り出す。data に含まれる var1 の値が 3 より大きい場合には”A”，var1 が 3 以下かつ var2 が”Y” の場合には”B”，それ以外の場合には”C” と置換する。置換されたデータは var3 に格納される。

- 引数:
 - 第 1 引数: `~` の左側に置換条件を指定し、右側に置換値（文字列でも数値でも可）を指定する。
`&` や `|` を用いて条件を複数並べても良い
 - 第 2 引数以降: 必要な場合には、第 1 引数に倣って順次置換条件と置換値を指定する。
 - 最終引数: すべての置換条件に合致しない場合の置換値を指定する場合、`~` の左側に `TRUE` を指定し、右側に置換値を指定する。もしこの指定がなかった場合、すべての条件に合致しないものは NA に置換される。
- 返り値: 置換後の変数
- 留意点: `if_else()` と `recode()` の良いところ取りをした関数。複数の置換条件を設ける時に `if_else()` のように入れ子にする必要がなく、`recode()` のように条件を並べて書くだけで良い。また、`recode()` のように文字列からの置換に限定されておらず、`if_else()` のように文字列、数値いずれの置換にも対応している。`mutate()` の中で用いる場合には、`data$` をつける必要はない。

5.4 ワイド型データからロング型データへの変換

- 例文: `data.long<-pivot_longer(data, cols=c(var1, var2), names_to=" 変数名", values_to=" 値")`
- 引数:
 - 第 1 引数: 変換したいデータフレームオブジェクト
 - 第 2 引数: 1 変数にまとめた複数の変数を `c()` を用いて指定する。 `c(-var1, -var2)` というように `-` を用いて指定した場合、指定した変数以外の変数がすべて使われたロング型データが作成される。
 - 第 3 引数 `names_to=" 変数名"`: ロング型に変換した際の変数名を指定する。

- 第4引数 `values_to=" 値"`: ロング型に変換した際の値が格納される変数名を指定する.
- 返回值: ロング型に変換されたデータフレームオブジェクト.
- 留意点: 指定していない変数は同じ値が繰り返される. 例えば, 以下の例では, `id` と性別は `変数 = 変数 1` と `変数 = 変数 2` とで同じ値が繰り返される.

5.5 ロング型データからワイド型データへの変換

- 例文: `data_wide<-pivot_wider(data, names_from=" 変数名", values_from=" 値")`
- 引数:
 - 第1引数: 変換したいデータフレームオブジェクト
 - 第2引数 `names_from=" 変数名"`: ワイド型に変換した際の変数名を指定する.
 - 第3引数 `values_from=" 値"`: ワイド型に変換した際の値を指定する.
- 返回值: ワイド型に変換されたデータフレームオブジェクト

6 データの記述統計量を得る

6.1 1変数の単純な記述統計量を得る

6.1.1 数値ベクトルの平均値を求める.

- 例文: `mean(data, na.rm=TRUE)`
- 引数:
 - 第1引数: 数値ベクトル
 - 第2引数 `na.rm=TRUE`: 欠損値を除外する. 省略した場合 `FALSE` となり, 欠損値が含まれる場合には `NA` が返される.

6.1.2 数値ベクトルの標準偏差を求める.

- `sd()`
- 例文: `sd(data, na.rm=TRUE)`
- 引数:
 - 第1引数: 数値を含むオブジェクト
 - 第2引数 `na.rm=TRUE`: 欠損値を除外する. 省略した場合 `FALSE` となり, 欠損値が含まれる場合には `NA` が返される.

6.1.3 数値ベクトルの分散を求める.

- 例文: `var(data, na.rm=TRUE)`

- 引数:
 - 第 1 引数: 数値ベクトル
 - 第 2 引数 `na.rm=TRUE`: 欠損値を除外する. 省略した場合 `FALSE` となり, 欠損値が含まれる場合には `NA` が返される.

6.1.4 数値ベクトルのの最大値を求める.

- 例文: `max(data)`
- 引数:
 - 第 1 引数: 数値ベクトル
 - 第 2 引数 `na.rm=TRUE`: 欠損値を除外する. 省略した場合 `FALSE` となり, 欠損値が含まれる場合には `NA` が返される.

6.1.5 数値ベクトルのの最小値を求める.

- 例文: `min(data)`
- 引数:
 - 第 1 引数: 数値ベクトル
 - 第 2 引数 `na.rm=TRUE`: 欠損値を除外する. 省略した場合 `FALSE` となり, 欠損値が含まれる場合には `NA` が返される.

6.1.6 数値ベクトルのの中央値を求める.

- 例文: `median(data)`
- 引数:
 - 第 1 引数: 数値ベクトル
 - 第 2 引数 `na.rm=TRUE`: 欠損値を除外する. 省略した場合 `FALSE` となり, 欠損値が含まれる場合には `NA` が返される.

6.1.7 数値ベクトルのの分位数を求める.

- 例文: `quantile(data, c(0.25, 0.75))`
- 引数:
 - 第 1 引数: 分位数を求めたい数値を含むオブジェクト
 - 第 2 引数: 分位数の位置を 0 から 1 で指定する. 複数のものを同時に得たい場合には `c(0.25, 0.75)` のように指定する.
- 返り値: 指定した分位数の値を含むベクトル (名前付き).

6.1.8 度数分布を求める.

- 例文: `table(data)`, `table(data$factor1, data$factor2)`
- 引数: 含まれている値の頻度を求めたいオブジェクト
- 返り値: 水準ごとの頻度を含むテーブルオブジェクト.
- 留意点: 引数に複数個の変数を指定した場合, それらのクロス集計表が作成される.

6.1.9 1 変数の標準誤差を得る

`sd()` を用いて標準偏差を求め, その値を**標本数の平方根**で割ることで求める.

```
num <- length(data)
data_res <- sd(data) / sqrt(num)
```

6.1.10 1 変数の平均値の信頼区間を得る

- 例文: `t.test(data, conf.level = 0.95)$conf.int`
- 引数:
 - 第 1 引数: 数値を含むオブジェクト
 - 第 2 引数 `conf.level=0.95` (省略可): 信頼区間の信頼係数を指定する. 省略した場合は 0.95.
- 返り値: 信頼区間の値
- 留意点: `t.test()` を用いて **1 変数の t 検定** (母平均が指定した値 (デフォルトでは 0) でないかの検定) を行った後, 出力されるオブジェクトに含まれる信頼区間 `conf.int` の値を `$` で得る.

6.2 グループごとの記述統計量を得る

`aggregate()` を用いる.

6.2.1 1 つの変数に対して

- 例文: `aggregate(var1 ~ group, data, mean)`
- 引数:
 - 第 1 引数: `~` の左側には集計したい変数 (`var1`) を, 右側にはグループ分けのための要因型変数 (`group`) を指定する.
 - 第 2 引数: データフレームオブジェクト.
 - 第 3 引数: 集計関数を指定する. `mean` や `sd` など.
- 返り値: グループごとに集計された結果が, 与えた `group` 変数の名前と `var1` 変数の名前を持った列を持つデータフレームとして返される.

6.2.2 複数の変数に対して

- 例文: `aggregate(cbind(var1, var2) ~ group, data, mean)`
- 引数:
 - 第 1 引数: `~` の左側に集計したい複数の変数を `cbind()` を用いて指定する。右側にはグループ分けのための要因型を指定する。
 - 第 2 引数: データフレームオブジェクト。
 - 第 3 引数: 集計関数を指定する。 `mean` や `sd` など。
- 返回值: グループごとに集計された各変数の結果がデータフレームとして返される。

6.2.3 1 つの変数に対して、複数のグループ分け基準を指定して

- 例文: `aggregate(var1 ~ group1 + group2, data, mean)`
- 引数:
 - 第 1 引数: `~` の左側に集計したい変数を指定する。右側にはグループ分けのため複数の要因型変数を `+` を用いて並べる。
 - 第 2 引数: データフレームオブジェクト。
 - 第 3 引数: 集計関数を指定する。 `mean` や `sd` など。
- 返回值: 複数のグループでクロス集計された結果がロング型のデータフレームとして返される。

6.2.4 複数の変数に対して、複数のグループ分け基準を指定して

- 例文: `aggregate(cbind(var1, var2) ~ group1 + group2, data, mean)`
- 引数:
 - 第 1 引数: `~` の左側に集計したい複数の変数を `cbind()` を用いて指定する。右側にはグループ分けのため複数の要因型変数を `+` を用いて並べる。
 - 第 2 引数: データフレームオブジェクト。
 - 第 3 引数: 集計関数を指定する。 `mean` や `sd` など。
- 返回值: 複数のグループでクロス集計された結果がロング型のデータフレームとして返される。

6.2.5 複数の集計関数を同時に適用

- 例文: `aggregate(var1 ~ group, data, function(x) c(mean=mean(x), sd=sd(x)))`
- 引数:
 - 第 1 引数: `~` の左側には集計したい変数名を、右側にはグループ分けのための要因型を指定する。`cbind()` を使って複数の変数を指定することも可能。
 - 第 2 引数: データフレームオブジェクトを指定する。 `+` を使って複数の変数を指定することも可能。
 - 第 3 引数: `function(x)` と置いたうえで半角スペースを置き、 `c()` で集計関数を与える。この際、名前付きベクトルで与えると、出力結果にその名称が反映される。

7 比較をしたい

7.1 2群で平均値を比較したい

`t.test()` を用いる。データの対応関係の有無で使い方が異なる。

7.1.1 データに対応関係がない場合

- 例文: `t.test(x=data1, y=data2)`
- 引数:
 - 第1引数 `x`: 比較したい数値ベクトル 1. `x=` は省略しても良い
 - 第2引数 `y`: 比較したい数値ベクトル 2. `y=` は省略しても良い
 - 第3引数 `alternative`: 省略可。省略した場合、両側検定を行う。片側検定を行いたい場合には `"greater"` または `"less"` を指定する。
- 返回值: 検定結果を含むリストオブジェクト。 `print()` で出力させることができる。

ロング型データの場合、次のように記述することもできる。

- 例文: `t.test(var~group, data)`
- 引数:
 - 第1引数: 数値変数とグループ変数 `~` で結んで指定する。グループ変数は要因型で水準は2つだけであることが前提。
 - 第2引数: データフレームオブジェクト

7.1.2 データに対応関係がある場合

- 例文: `t.test(x=data1, y=data2, paired=TRUE)`
- 引数:
 - 第1引数 `x`: 比較したい数値ベクトル 1. `x=` は省略しても良い
 - 第2引数 `y`: 比較したい数値ベクトル 2. `y=` は省略しても良い
 - 第3引数 `alternative`: 省略可。省略した場合、両側検定を行う。片側検定を行いたい場合には `"greater"` または `"less"` を指定する。
 - 第4引数 `paired=TRUE`: 対応関係がある場合には `TRUE` を指定する。
- 留意点: 対応関係がある場合、データの数等は等しくなければならない。また、対応関係のあるデータの場合、ロング型データを用いることはできない。あくまで `filter()` などを用いて、それぞれのデータを別個のベクトルとして抽出してから `t.test()` を適用する。

7.2 2群で分散を比較したい

- 例文: `var.test(x=data1, y=data2)`
- 引数:
 - 第1引数 `x`: 比較したい数値ベクトル 1. `x=` は省略しても良い
 - 第2引数 `y`: 比較したい数値ベクトル 2. `y=` は省略しても良い
- 返回值: 検定結果を含むリストオブジェクト. `print()` で出力させることができる.

7.3 2群で比率を比較したい

- 例文: `prop.test(c(10, 20), c(100, 200), correct=FALSE)`
- 引数:
 - 第1引数: それぞれの群での成功数（分子に来る数）をベクトルで指定する.
 - 第2引数: それぞれの群での総数（分母に来る数）をベクトルで指定する.
 - 第3引数 `correct=FALSE`: Yates の補正を行うかどうか. 省略すると `TRUE`. 5 以上のサンプルがあるのであれば `FALSE` にする.
- 返回值: 検定結果を含むリストオブジェクト. `print()` で出力させることができる.
- 留意点: 最近ではサンプル数が 5 以下の時は, 以下で説明している Fisher の正確確率検定を用いることが多いので, `correct=FALSE` を付けるが基本となる.

予め `table()` を用いてクロス集計表を作成しておくと, それを引数として `prop.test()` を適用することもできる.

- 例文: `prop.test(data_table)`
- 引数: `table()` で作成したクロス集計表オブジェクト. 関数の中で `table()` を使ってもよい.

7.4 2つの要因の独立性を検定したい

クロス集計表を作成したときに, いずれかのセルが 5 以下になっている場合には, Fisher の正確確率検定を行う. それ以外の場合には, カイ二乗検定を行う (Fisher の正確確率検定を使っても良いが, 計算時間が長くなってしまう, 処理がタイムアウトしてしまう可能性がある).

7.4.1 カイ二乗検定

- 例文: `chisq.test(data_table, correct=FALSE)`
- 引数:
 - 第1引数: `table()` を用いて作成したクロス集計表オブジェクト. `chisq.test()` の中で `table()` を使ってもよい.

- 第2引数 `correct=FALSE`: Yates の補正を行うかどうか. 省略すると `TRUE`. 5 以上のサンプルがあるのであれば `FALSE` にする.
- 返回值: 検定結果を含むリストオブジェクト. `print()` で出力させることができる.
- 留意点: 最近ではサンプル数が5以下の時は, 以下で説明している Fisher の正確確率検定を用いることが多いので, `correct=FALSE` を付けるが基本となる. 2×2 のクロス集計表の場合, `prop.test()` と同じ結果が出力される.

`table()` を介さず, それぞれの要因のベクトルデータを直接 `chisq.test()` に与えても良い.

- 例文: `chisq.test(data$factor1, data$factor2, correct=FALSE)`
- 引数:
 - 第1引数: 要因1のベクトルデータ
 - 第2引数: 要因2のベクトルデータ
 - 第3引数 `correct=FALSE`: Yates の補正を行うかどうか. 上記参照.

7.4.2 Fisher の正確確率検定

f- 例文: `fisher.test(data_table)` - 引数: - 第1引数: `table()`を用いて作成したクロス集計表オブジェクト. `fisher.test()` の中で `table()` を使ってもよい.

7.5 3群以上での比較をしたい

平均の比較にせよ, 分散の比較にせよ, 比率の比較にせよ, 3群以上の間で比較をしたい場合には, 予めすべての組み合わせでの一対比較を行ない, p 値を取得したうえで, `p.adjust()` を用いて多重比較の補正を行う.

- 例文: `p.adjust(c(p1, p2, p3), method="holm")`
- 引数:
 - 第1引数: 比較したい p 値をベクトルで指定する.
 - 第2引数 `method`: 多重比較の補正方法を指定する. `"holm"` や `"bonferroni"` など.
- 留意点: p 値はいずれの検定の場合も, 出力されるオブジェクトに `$p.value` でアクセスすることで取得できる.

8 相関を調べたい

8.1 相関係数を求めたい

- 例文: `cor(data1, data2)`, `cor(data)`
- 引数:
 - 第1引数: 数値ベクトル1, もしくは数値のデータフレーム

- 第 2 引数: 数値ベクトル 2
- 返回值: 相関係数を返す.

8.2 相関係数の検定をしたい

- 例文: `cor.test(data1, data2)`
- 引数:
 - 第 1 引数: 数値ベクトル 1
 - 第 2 引数: 数値ベクトル 2
- 返回值: 相関係数の検定結果を含むリストオブジェクト.

8.3 相関の差の検定をしたい

8.3.1 独立サンプルにおける相関の差の検定

- 例文: `cocor.indep.groups(r1, r2, n1, n2)`
- `cocor` パッケージを用いる.
- 引数:
 - 第 1 引数: 相関係数 1
 - 第 2 引数: 相関係数 2
 - 第 3 引数: サンプルサイズ 1
 - 第 4 引数: サンプルサイズ 2
- 返回值: 相関係数の差の検定結果を含むリストオブジェクト. 検定結果は中段の `fisher1925` の部分にあり, この中の `p-value` が有意確率である.
- Fisher の Z 検定

8.3.2 同一サンプルにおける 1 つの変数 J が共通している相関の差の検定

- 例文: `cocor.dep.groups.overlap(r.jk, r.jh, r.hk, n)`
- `cocor` パッケージを用いる.
- 引数:
 - 第 1 引数 `r.jk`: 相関係数比較元
 - 第 2 引数 `r.jh`: 相関係数比較先
 - 第 3 引数 `r.hk`: 共通していない変数同士の相関係数
 - 第 4 引数: サンプルサイズ
- 返回值: 相関係数の差の検定結果を含むリストオブジェクト. 検定結果は `steiger1980` の結果を見るのが一般的である (この検定のことを Stieger の Z 検定と呼ぶ)

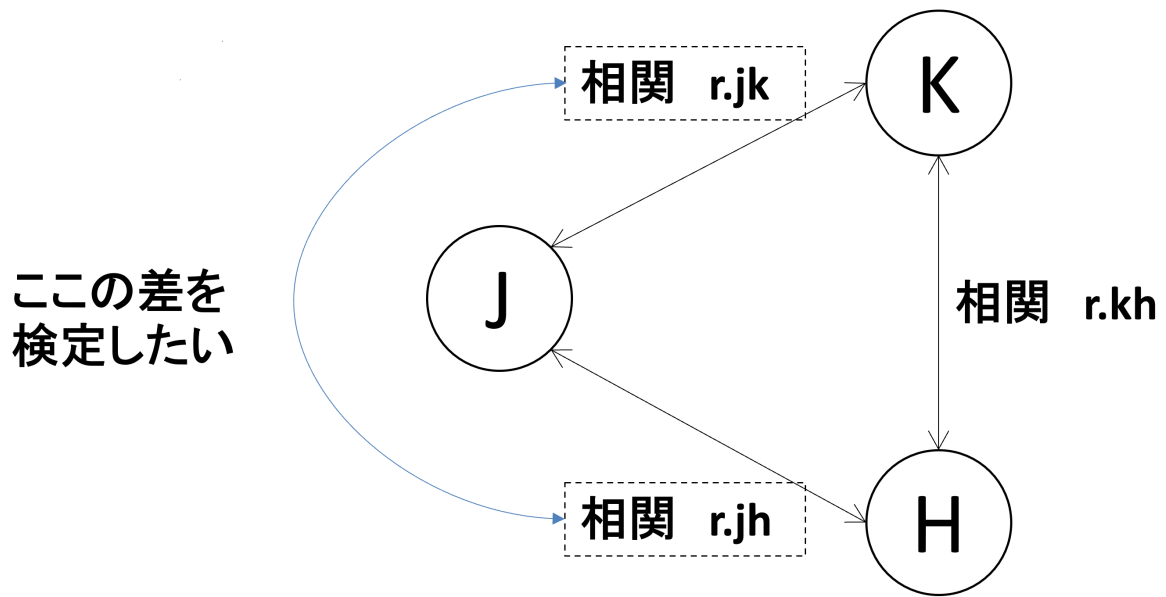


図1 1つの変数が共通している相関の差の検定

8.3.3 同一サンプルにおける互いに異なる変数間の相関の差の検定

- 例文: `cocor.dep.groups.nonoverlap(r.jk, r.hm, r.jh, r.jm, r.kh, r.km n)`
- `cocor` パッケージを用いる.
- 引数:
 - 第1引数 `r.jk`: 1つ目 (J-K) の相関係数
 - 第2引数 `r.hm`: 2つ目 (H-M) の相関係数
 - 第3引数 `r.jh`: J-H 間の相関係数
 - 第4引数 `r.jm`: J-M 間の相関係数
 - 第5引数 `r.kh`: K-H 間の相関係数
 - 第6引数 `r.km`: K-M 間の相関係数
 - 第7引数 `n`: サンプルサイズ
- 返り値: 相関係数の差の検定結果を含むリストオブジェクト. 検定結果は `steiger1980` の結果を見るのが一般的である (この検定のことを `Stieger` の `Z` 検定と呼ぶ)

9 連続変数間での予測式を作りたい・因果関係を調べたい (回帰分析)

9.1 単純な重回帰分析

- 例文: `lm(y ~ x1 + x2, data)`
- 引数:

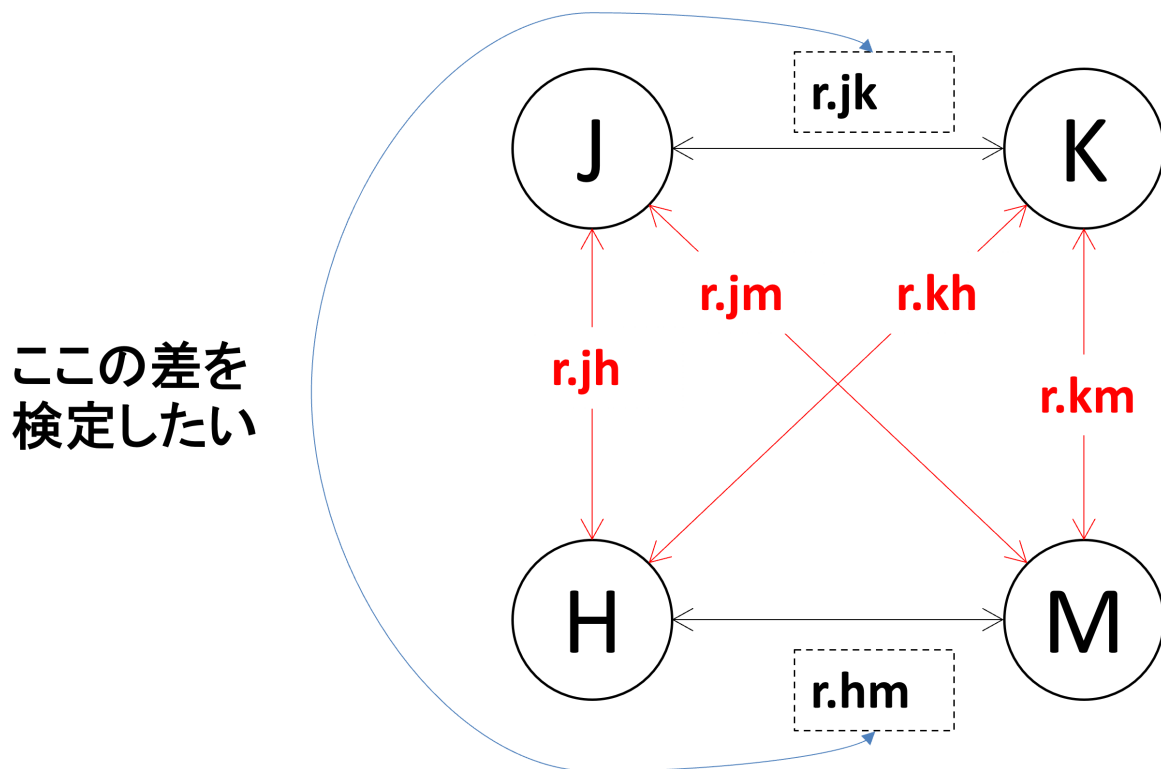


図2 互いに異なる変数間の相関の差の検定

- 第 1 引数: 目的変数（従属変数）と説明変数（独立変数）を `~` で結んで指定する。
- 第 2 引数: データフレームオブジェクト
- 返回值: 回帰分析の結果を含むオブジェクト。 `summary()` で出力させることができる。

9.2 標準化重回帰分析

- 例文: `lm(scale(y) ~ scale(x1) + scale(x2), data)`
- 引数:
 - 第 1 引数: 目的変数（従属変数）と説明変数（独立変数）を `~` で結んで指定する。このとき、標準化したい変数には `scale()` を適用すると、その変数が標準化された上で回帰分析が掛けられる。
 - 第 2 引数: データフレームオブジェクト
- 返回值: 回帰分析の結果を含むオブジェクト。 `summary()` で出力させることができる。
- 留意点: 全変数を標準化する場合には、データそのものに `scale()` を適用することもできる。ただし、その場合には `scale()` の返回值が行列型になるため、データフレームオブジェクトとして扱うためには `as.data.frame()` を適用する必要がある。

9.3 交互作用項を含む重回帰分析

- 例文: `lm(y ~ x1 * x2, data)`
- 引数:
 - 第 1 引数: 目的変数（従属変数）と説明変数（独立変数）を `~` で結んで指定する。交互作用項を含む場合には `*` を用いて指定する。なお、交互作用と主効果を明確に分けて表現したい場合には、`x1+x2+x1:x2` と記述すれば良い。
 - 第 2 引数: データフレームオブジェクト
- 返回值: 回帰分析の結果を含むオブジェクト。summary() で出力させることができる。

9.4 一部の変数をカテゴリカル変数として扱いたい

- 例文: `lm(y ~ factor(x1) + x2, data)`
- 引数:
 - 第 1 引数: 目的変数（従属変数）と説明変数（独立変数）を `~` で結んで指定する。カテゴリカル変数には `factor()` を適用する。
 - 第 2 引数: データフレームオブジェクト
- 返回值: 回帰分析の結果を含むオブジェクト。summary() で出力させることができる。

9.5 ステップワイズ法による変数選択

- 例文: `step(res.lm, trace=FALSE)`
- 引数:
 - 第 1 引数: 回帰分析の結果を含むオブジェクト。
 - 第 2 引数 `trace`（省略化）: 途中経過を表示するかどうか。FALSE で非表示となる。省略した場合は TRUE となり、変数選択の過程がすべて表示される。
 - 第 3 引数 `scope=list(lower=)`（省略可）: 変数選択の範囲を指定する。lower には最小モデル（削除してほしくない変数のみで実施された lm() の結果）を指定する。
 - 第 4 引数 `direction`（省略化）: 変数選択の方向を指定する。"both" で前進選択と後退選択を同時に行う。他には "forward"（前進のみ）や "backward"（後退のみ）がある。省略すると "both" となる。
- 返回值: ステップワイズ法による変数選択の結果を含む lm オブジェクト。summary() で出力させることができる。
- 留意点: 変数を入れたり外したりしながら lm() を繰り返し実施し、AIC が最も低くなるようなモデルを見つけ出す。あくまで理論は一切考慮せずに結果だけを見て変数選択するため、その結果が理論的に良いモデルであるとは限らない。

10 離散変数と連続変数の間の因果関係を調べたい（分散分析）

10.1 1 要因分散分析

10.1.1 対応なしデータ/被験者間計画

- 例文: `aov(y ~ x, data)`
- 引数:
 - 第 1 引数: モデル式. 目的変数（従属変数）と説明変数（独立変数）を `~` で結んで指定する. **説明変数は要因型である必要がある.**
 - 第 2 引数: データフレームオブジェクト
- 返回值: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

10.1.2 対応ありデータ/被験者内計画

2つの書き方がある.

■10.1.2.1 その 1

- 例文: `aov(y ~ x + Error(subject/x), data)`
- 引数:
 - 第 1 引数: モデル式. 目的変数（従属変数）と説明変数（独立変数）を `~` で結んで指定する. 各項の説明は以下の通り.
 - * `y`: 目的変数（連続変数）
 - * `x`: 説明変数（要因型変数）
 - * `Error(subject/x)`: `subject`（被験者 ID もしくはサンプル ID.）の違いによる影響（`Subject` の主効果, `Sucject` と `x` の交互作用）を誤差として扱うことを指示する項.
 - ・ もし `x` の各水準で各 `Sucject` の測定が 1 回しかない場合には, `Sucject` と `x` の交互作用は検討できないので `Error(subject)` と記述しても同じ結果が得られる.
 - 第 2 引数: データフレームオブジェクト
- 返回值: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

■10.1.2.2 その 2

- 例文: `aov_ez(id=subject, dv=y, within=x, data=data)`
- `afex` パッケージを用いる.

- 引数:
 - 第 1 引数 `id`: 被験者 ID もしくはサンプル ID.
 - 第 2 引数 `dv`: 目的変数 (従属変数)
 - 第 3 引数 `within`: 説明変数 (要因型変数)
 - 第 4 引数 `data`: データフレームオブジェクト
 - 第 5 引数 `type=3`: 分散分析の種類. "3" で被験者内計画の分散分析を行う
- 返り値: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

10.2 2 要因分散分析

10.2.1 対応なしデータ/被験者間計画

2 つの要因が共に被験者間計画になっている場合である. 2 つの方法がある..

■10.2.1.1 その 1 `aov()` と `Anova()` を組み合わせて行う.

- 例文:

```
library(car)
res.aov <- aov(y ~ x1*x2, data)
Anova(res.aov, type=3)
```

- `car` パッケージを用いる (`Anova()` に対して).
- 引数:
 - `aov()` 部分について
 - * 第 1 引数: モデル式. 目的変数と 2 つの説明変数 (共に**要因型**) を `~` で結んで指定する. 交互作用項を含む場合には両説明変数 `*` を用いて並べるか, `+` で並べたあとに `x1:x2` という交互作用項を置く.
 - `y`: 目的変数 (連続変数)
 - `x1`: 説明変数 (要因型変数) その 1
 - `x2`: 説明変数 (要因型変数) その 2
 - `x1:x2`: 交互作用項. `x1*x2` と記述した場合には記述不要.
 - * 第 2 引数: データフレームオブジェクト
 - `Anova()` 部分について
 - * 第 1 引数: `aov()` の結果を含むオブジェクト
 - * 第 2 引数 `type=3`: 交互作用項を持つ 2 要因分散分析を行う場合には `type=3` を指定する. 交互作用をおかない場合には `type=2` を指定する.
- 返り値: 結果を含むオブジェクト.

- 留意点: 2 要因分散分析を行う場合には, `aov()` の結果を `Anova()` に与えることで, 2 要因分散分析を行うことができる。

■10.2.1.2 その2

- 例文: `aov_ez(id=subject, dv=y, between=c(x1, x2), data=data, type=3)`
- `afex` パッケージを用いる。
- 引数:
 - 第 1 引数 `id`: 被験者 ID もしくはサンプル ID.
 - 第 2 引数 `dv`: 目的変数 (従属変数)
 - 第 3 引数 `between`: 2 つの説明変数 (要因型変数) を `c()` で括って指定する。
 - 第 4 引数 `data`: データフレームオブジェクト
 - 第 5 引数 `type=3`: 分散分析の種類. "3" で被験者内計画の分散分析を行う
- 返回值: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる。
- 留意点: `aov_ez()` を用いる場合, サンプル識別番号 `subject` (要因型) を設ける必要がある。被験者間計画なので, サンプル識別番号はあくまで 1 つのデータと 1:1 で対応することになる。

10.2.2 対応ありデータ/被験者内計画

2 つの要因が共に被験者内計画になっている場合である。2 つの方法がある。

■10.2.2.1 その1

- 例文: `aov(y ~ x1*x2 + Error(subject/(x1*x2)), data)`
- 引数:
 - 第 1 引数: モデル式. 目的変数 (従属変数) と説明変数 (独立変数) を `~` で結んで指定する。各項の説明は以下の通り。
 - * `y`: 目的変数 (連続変数)
 - * `x1`: 説明変数 (要因型変数) その 1
 - * `x2`: 説明変数 (要因型変数) その 2
 - * `x1:x2`: 交互作用項. `x1*x2` と記述した場合には記述不要。
 - * `Error(subject/(x1*x2))`: `subject` (被験者 ID もしくはサンプル ID.) の違いによる影響 (Subject の主効果, Subject と x の交互作用, Subject と y の交互作用, Subject と x と y の交互作用) を誤差として扱うことを指示する項。
 - 第 2 引数: データフレームオブジェクト
- 返回值: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる。

■10.2.2.2 その2

- 例文: `aov_ez(id=subject, dv=y, within=c(x,x2), data=data, type=3)`
- `afex` パッケージを用いる.
- 引数:
 - 第1引数 `id`: 被験者 ID もしくはサンプル ID.
 - 第2引数 `dv`: 目的変数 (従属変数)
 - 第3引数 `within`: 2つの説明変数 (要因型変数) を `c()` で括って指定する.
 - 第4引数 `data`: データフレームオブジェクト
 - 第5引数 `type=3`: 分散分析の種類. "3" で被験者内計画の分散分析を行う
- 返り値: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

10.2.3 一方の変数だけ対応ありデータ/混合計画

2つの要因のうち1つは被験者間計画, もう一つが被験者内計画になっている場合である. 2つの方法がある.

■10.2.3.1 その1

- 例文: `aov(y ~ x1*x2 + Error(subject/x2), data)`
- 引数:
 - 第1引数: モデル式. 目的変数 (従属変数) と説明変数 (独立変数) を `~` で結んで指定する. 各項の説明は以下の通り.
 - * `y`: 目的変数 (連続変数)
 - * `x1`: 被験者間計画の説明変数 (要因型変数)
 - * `x2`: 被験者内計画の説明変数 (要因型変数)
 - * `x1:x2`: 交互作用項. `x1*x2` と記述した場合には記述不要.
 - * `Error(subject/x2)`: `subject` (被験者 ID もしくはサンプル ID.) の違いによる影響 (Subject の主効果, Subject と `x` の交互作用, Subject と `y` の交互作用, Subject と `x` と `y` の交互作用) を誤差として扱うことを指示する項.
 - 第2引数: データフレームオブジェクト
- 返り値: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

■10.2.3.2 その2

- 例文: `aov_ez(id=subject, dv=y, between=x1, within=x2, data=data, type=3)`

- `afex` パッケージを用いる.
- 引数:
 - 第 1 引数 `id`: 被験者 ID もしくはサンプル ID.
 - 第 2 引数 `dv`: 目的変数 (従属変数)
 - 第 3 引数 `between`: 被験者間計画の説明変数 (要因型変数)
 - 第 4 引数 `within`: 被験者内計画の説明変数 (要因型変数)
 - 第 5 引数 `data`: データフレームオブジェクト
 - 第 6 引数 `type=3`: 分散分析の種類. `"3"` で被験者内計画の分散分析を行う
- 返回值: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

11 変数を集約したい

11.1 主成分分析

- 例文: `prcomp(data, scale.=TRUE)`
- 引数:
 - 第 1 引数: データフレームオブジェクト
 - 第 2 引数 `scale=TRUE`: データを標準化するかどうか. 標準化したくない場合には `FALSE` を指定する. 省略すると `FALSE` となる. 通常は標準化することが望ましいとされる.
- 返回值: 主成分分析の結果を含むオブジェクト. 各主成分の分散説明率や累積説明率は `summary()` で出力させることができる.

11.1.1 主成分負荷量を得たい

- 例文: `res.pca$rotation`
- 引数: 主成分分析の結果を含むオブジェクトに `$` をつけて `rotation` を指定する.
- 返回值: 主成分負荷量を含む行列オブジェクト.

11.1.2 主成分得点を得たい

- 例文: `res.pca$x`
- 引数: 主成分分析の結果を含むオブジェクトに `$` をつけて `x` を指定する.
- 返回值: 主成分得点を含む行列オブジェクト.

11.2 因子分析

11.2.1 因子数を模索したい

■11.2.1.1 スクリーンプロットを得たい

- 例文: `fa.parallel(data, fa="fa")`
- `psych` パッケージを用いる.
- 引数:
 - 第 1 引数: データフレームオブジェクト
 - 第 2 引数 `fa="fa"` (省略可): 出力を因子分析用にするか, 主成分分析用にするかを指定する. 因子分析用にする場合には `"fa"` を指定する. 主成分分析用にする場合には `"pc"` を指定する. 省略すると両方が表示される.
- 返回值: スクリーンプロットが出力されるとともに, 因子数の候補が表示される.

■11.2.1.2 vss 法を実施したい

- 例文: `vss(data)`
- `psych` パッケージを用いる.
- 引数:
 - 第 1 引数: データフレームオブジェクト
 - 第 2 引数 `n=8` (省略可): シミュレートするときの最大因子数を指定する. 省略すると 8 が指定される.
- 返回值: VSS の結果が表示されるとともに, VSS のグラフが出力される.

11.2.2 探索的因子分析を実施したい

- 例文: `fa(data, 2, rotate="oblimin", fm="minres")`
- `psych` パッケージを用いる.
- 引数:
 - 第 1 引数: データフレームオブジェクト
 - 第 2 引数: 因子数を指定する.
 - 第 3 引数 `rotate="oblimin"` (省略可): 因子負荷量の回転方法を指定する. 省略すると斜交回転の 1 種である `oblimin` が行われる. 他に, 直交回転系として `"varimax"`, `"quartimax"`, 斜交回転系で `"promax"` などが指定できる.

- 第 4 引数 `fm="minres"` (省略可): 因子分析の方法を指定する. `"minres"` で最小残差法が指定される. 他には `"ml"` (最尤法) や `"pa"` (主軸法) などが指定できる.
- 返回值: 因子分析の結果を含むオブジェクト. 出力は以下を参照.

11.2.3 因子分析の結果を出力させたい

- 例文: `print(res.fa, sort=TRUE, digits=2)`
- `psych` パッケージを用いる.
- 引数:
 - 第 1 引数: 因子分析の結果を含むオブジェクト
 - 第 2 引数 `sort=TRUE` (省略可): 因子負荷量を表示する際に, 各因子の因子負荷量の大きい順に表示するかどうか. `TRUE` にすると各因子ごとに負荷量の大きい順に表示される. 省略すると `FALSE` となり, 変数名の順に表示される.
 - 第 3 引数 `digits=2` (省略可): 因子負荷量の表示桁数を指定する. 省略すると 2 桁になる.
- 返回值: 因子分析の結果が表示される.

11.2.4 因子分析の結果を図示したい

- 例文: `fa.diagram(res.fa, digits=2, cut=.3, simple=TRUE)`
- `psych` パッケージを用いる.
- 引数:
 - 第 1 引数: 因子分析の結果を含むオブジェクト
 - 第 2 引数 `digits=2` (省略可): 因子負荷量の表示桁数を指定する. 省略すると 2 桁になる.
 - 第 3 引数 `cut=.3` (省略可): 因子負荷量の絶対値がこの値より小さい場合には表示しない. 省略すると 0.3 が指定される.
 - 第 4 引数 `simple=TRUE` (省略可): `TRUE` にすると各項目につき因子負荷量が最も高い因子との結びつきのみが表示される. `FALSE` にすると全ての因子との結びつきが表示する. ただしいずれも `cut` を下回るものについては表示されない.
- 返回值: 図示された因子分析の結果.

11.2.5 因子得点を得たい

- 例文: `res.fa$scores`
- 引数: 因子分析の結果を含むオブジェクトに `$` をつけて `scores` を指定する.
- 返回值: 因子得点を含む行列オブジェクト.

12 サンプルをグルーピングしたい

12.1 階層的クラスタリング法

以下の手順で実施する。

1. `scale()` を用いてデータを標準化する。
 - 引数: データフレームオブジェクト
2. `dist()` を用いてデータの距離行列を求める。
 - 第 1 引数: 標準化させたデータオブジェクト
 - 第 2 引数 `method`: 距離の計算方法を指定する。省略した場合、ユークリッド距離となる。
3. `hclust()` を用いて階層的クラスタリングを行う。
 - 第 1 引数: 距離行列オブジェクト
 - 第 2 引数 `method`: クラスタリングの方法を指定する。省略した場合には `"complete"` (最長距離法) が指定される。
4. `plot()` を用いてデンドログラムを表示させ、クラスタ数を決める。
5. `cutree()` を用いてクラスタリングの結果を取得する。
 - 第 1 引数: `hclust()` の結果を含むオブジェクト
 - 第 2 引数 `k`: クラスタ数を指定する。

例文:

```
data_std <- scale(data)
data_dist <- dist(data_std)
hc <- hclust(dist_data, method="ward.D2")
plot(hc)
res.clust<-cutree(hc, k=3)
```

- 最終的な返り値: 各サンプルに振られるクラスタ番号を収めたベクトル。これを例えば `data$cluster <- res.clust` のようにデータフレームに追加することで、クラスタリングの結果を元のデータフレームに保存することができ、クラスターごとの平均値の算出などに利用できる (参考)。

12.2 k-means 法

- 例文: `kmeans(data, centers=3, nstart=10)`
- 引数:
 - 第 1 引数: データフレームオブジェクト (標準化済みのものがよい)

- 第2引数 `centers`: クラスタ数を指定する.
- 第3引数 `nstart=10`: 初期値を変えてクラスタリングを行う回数を指定する.
- 返回值: クラスタリングの結果を含むオブジェクト. `res.kmeans$cluster` で各サンプルに振られるクラスタ番号を取得できる.
- 留意点: k-means 法はランダムに選択される初期値によってクラスタリングの結果が変わることがある. そのため, `nstart` を大きくすることで, 複数回クラスタリングを行い, 最もクラスタリングが良いものを選ぶことができる.

13 ggplot2 を用いたデータの可視化

tidyverse パッケージを（正確には ggplot2 パッケージ）読み込んでおくこと.

基本的に, 以下の流れで `+` で結び付けていく.

1. `ggplot()` でキャンバスオブジェクトを作成する.
2. `geom_` 系の描画関数を用いて, キャンバスオブジェクトに描画する.
3. x 軸や y 軸のラベル, タイトルなどを設定する.
4. 必要に応じた追加処理を行う.
5. キャンバスオブジェクトを出力する.

13.1 キャンバスの作成

- 例文: `ggplot(data, aes(x=x, y=y, color=group, fill=group, shape=group))`
- 引数:
 - 第1引数 `data` (省略可): キャンバスに結びつけるデータフレームオブジェクト. 省略した場合には描画関数内で指定する.
 - 第2引数 `aes()` (省略可): データに基づく描画要素を指定する. 以下のいずれも省略することができ, その場合には描画関数内の `aes()` で必要に応じて指定する. キャンバス作成時に指定すると, 以後の描画関数は全て, この設定の下で描画される. なお, 引数の順番は入れ替わっていても良い.
 - * 第1引数 `x` (省略可): x 軸に対応する変数を指定する.
 - * 第2引数 `y` (省略可): y 軸に対応する変数を指定する.
 - * 第3引数 `color` (省略可): 点や線について, データに基づいて色分けしたい場合, 色分けを指示する変数を指定する.
 - * 第4引数 `fill` (省略可): 塗りつぶしについて, データに基づいて塗り分けしたい場合, 色分けを指示する変数を指定する.
 - * 第5引数 `shape` (省略可): データに基づいてマーカーの形を変えたい場合, 形分けを指示する変数を指定する.
- 返回值: キャンバスオブジェクト

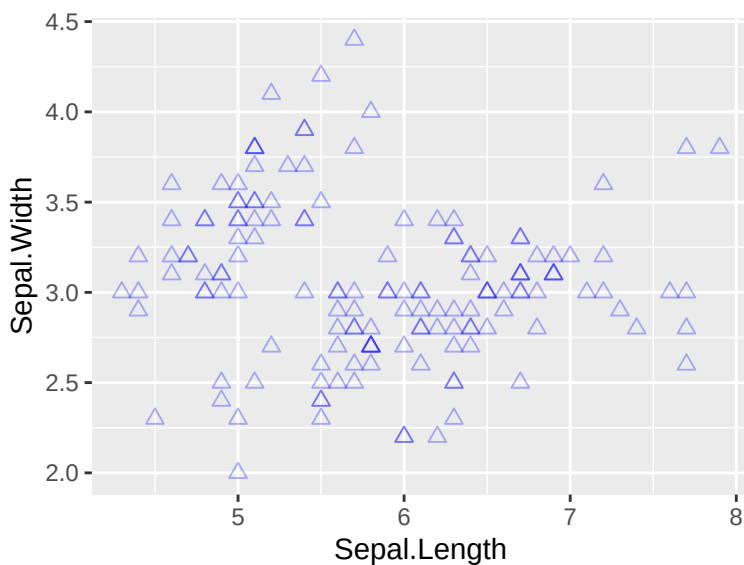
13.2 散布図を描く

- 例文: `geom_point(data,aes(x=x, y=y, shape=group), color="blue", size=2, alpha=0.5)`
- 引数:
 - 第 1 引数 `data` (省略可): キャンバスに結びつけるデータフレームオブジェクト. 先に `ggplot()` 内で指定している場合は省略可能.
 - 第 2 引数 `aes()` (省略可): 先に `ggplot()` 内で指定している場合は省略可能. 記載方法は先と同様.
 - 第 3 引数 `color` (省略可): 点の色を指定する. ここで色を指定すると `aes()` 内の色分け設定が無効化され, すべてがここで指定した色になる.
 - 第 4 引数 `size=2` (省略可): マーカーのサイズを指定する.
 - 第 5 引数 `alpha=0.5` (省略可): マーカーの透明度を指定する. 点の重なりを表現したい場合にはこれを指定するとよい.
 - 第 6 変数 `shape` (省略可): マーカーの形を数字で指定する. ここで形を指定すると `aes()` 内の形分け設定が無効化され, すべてがここで指定した形になる.
 - 第 7 引数 `position="jitter"` (省略可): “jitter” に設定すると, すべての点がデータの座標位置からランダムにずらされる. 省略するとデータの通りに重ね合わせて描画される.
- 返り値: 散布図が描かれた `ggplot` オブジェクト

```
library(ggplot2)
```

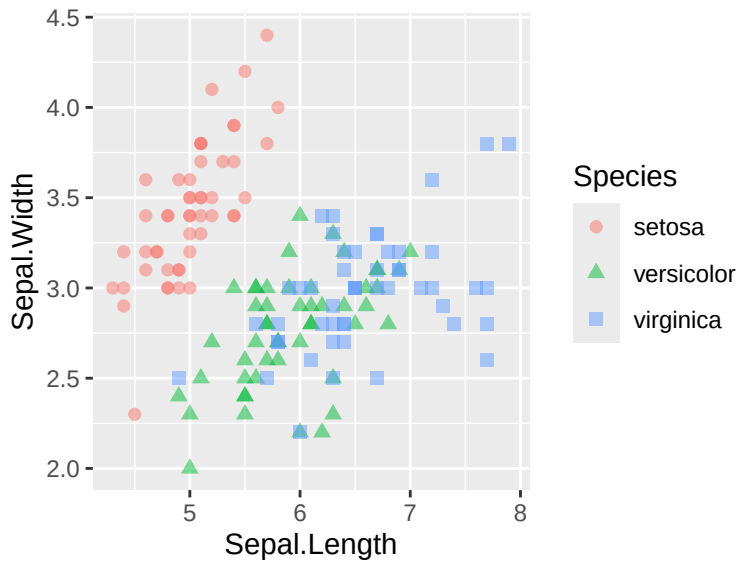
```
## 通常の散布図 (色を青に, 形を 2 (記号) に, 重なり対応として透明度を 0.3 に設定)
```

```
ggplot(data=iris, aes(x=Sepal.Length, y=Sepal.Width)) +  
  geom_point(color="blue", shape=2, size=2, alpha=0.3)
```



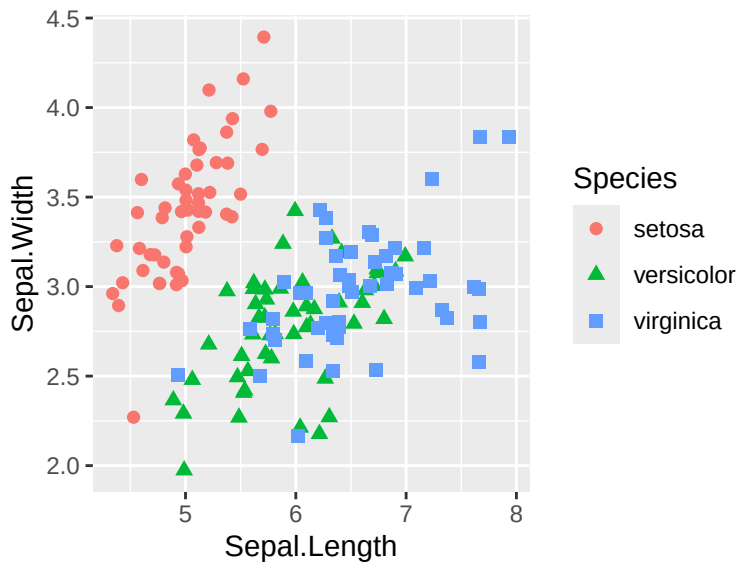
色分け設定をした散布図（重なり対応として透明度を 0.5 に設定

```
ggplot(data=iris, aes(x=Sepal.Length, y=Sepal.Width)) +  
  geom_point(aes(color=Species, shape=Species), size=2, alpha=0.5)
```



重なり対応として jitter を設定した散布図

```
ggplot(data=iris, aes(x=Sepal.Length, y=Sepal.Width)) +  
  geom_point(aes(color=Species, shape=Species), position="jitter", size=2)
```

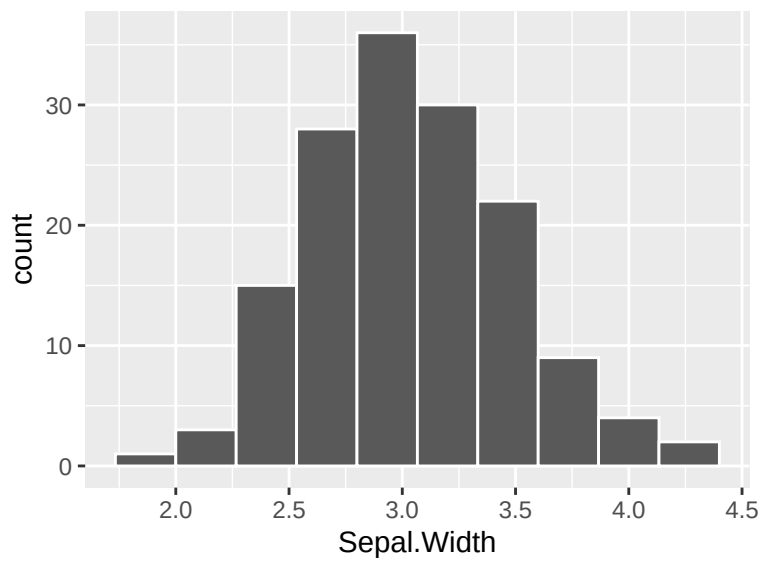


13.3 ヒストグラム（度数分布表）を描く

- 例文: `geom_histogram(data, aes(x=x, fill=group), bins=30, alpha=0.5)`
- 引数:
 - 第 1 引数 `data` (省略可): キャンバスに結びつけるデータフレームオブジェクト. 先に `ggplot()` 内で指定している場合は省略可能.
 - 第 2 引数 `aes()` (省略可): 先に `ggplot()` 内で指定している場合は省略可能. 留意点は以下の通り.
 - * 第 1 引数 `x` ないしは `y`: 度数分布を見たい変数を指定する. いずれか一方のみを指定する. `x` を指定すると縦方向のバーに, `y` を指定すると横方向のバーになる.
 - * 第 2 引数 `fill` (省略可): 設定した場合, グループごとに度数分布が作られた上で塗分けがなされる. 省略した場合はグループの区別なくすべてのデータを一まとめにした度数分布が作成される.
 - * 第 3 引数 `color` (省略可): バーの枠線がグループごとに色分けされる. その他の機能は `fill` と同様.
 - 第 3 引数 `bins=30` もしくは `binwidth=0.3`: 区間の数 (bins), もしくは区間幅 (binwidth) のいずれかを指定する. `bins` を指定した場合, 区間幅は `x` の最大値と最小値を `bins` の値で割った値になる.
 - 第 4 引数 `color` (省略可): 枠線の色を指定する. ここで色を指定すると `aes()` 内の色分け設定が無効化され, すべてがここで指定した色になる.
 - 第 5 引数 `fill` (省略可): バーの塗りつぶし色を指定する. ここで色を指定すると `aes()` 内の塗分け設定が無効化され, すべてがここで指定した色になる.
 - 第 6 引数 `alpha=0.5` (省略可): ヒストグラムの透明度を指定する. 省略した場合は透明度は 0 となる.
 - 第 7 引数 `position="dodge"` (省略可): `aes()` で `color` や `fill` を設定し, グループごとに塗分けする場合に, 各グループのバーの位置を指定する. “dodge” で横に並べる. “identity” で重ねる. “stack” で積み上げる. “fill” で積み上げ比率になる.
- 返り値: ヒストグラムが描かれた `ggplot` オブジェクト
- 留意点: グループ設定をした場合, `position` を “identity” に設定すると, 各グループのバーが重なってしまい, 描画順序によっては背の低いバーが背の高いバーに隠されてしまうため, 透明度を設定することが望ましい.

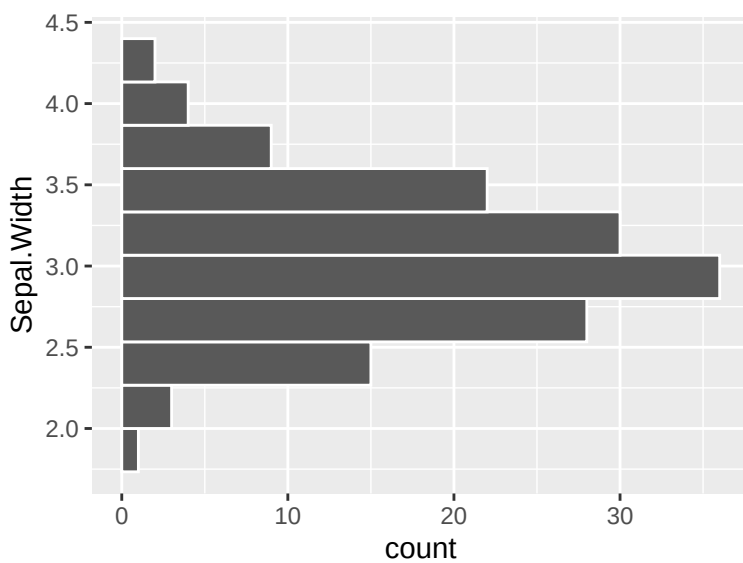
通常のヒストグラム（グループ設定なし）

```
ggplot(data=iris) +  
  geom_histogram(aes(x=Sepal.Width), color="white", bins=10, position="stack")
```



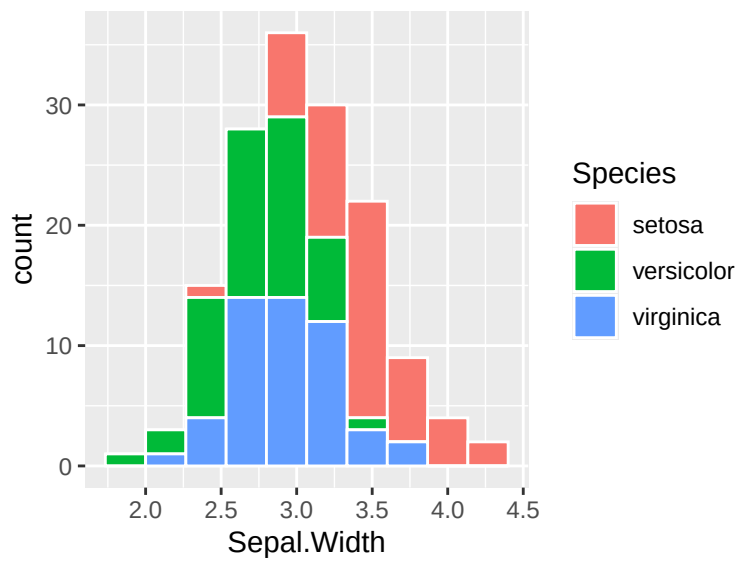
横棒グラフのヒストグラム（グループ設定なし）

```
ggplot(data=iris) +  
  geom_histogram(aes(y=Sepal.Width), color="white", bins=10, position="stack")
```



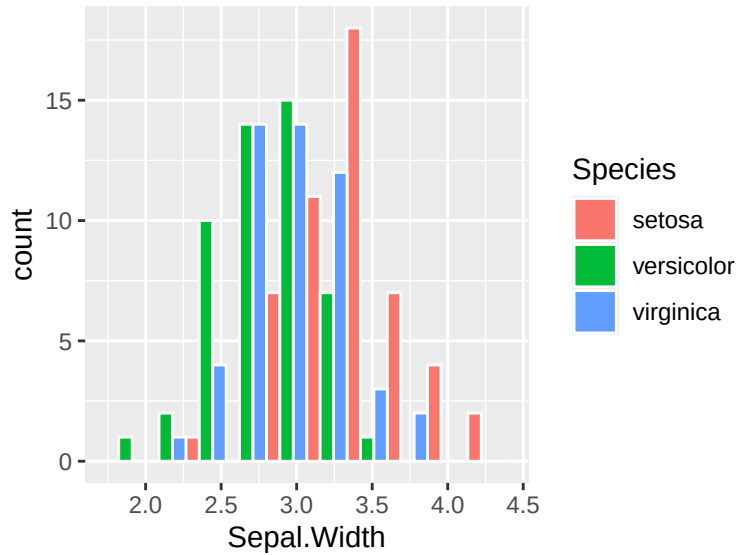
グループ設定ありの積み上げヒストグラム

```
ggplot(data=iris) +  
  geom_histogram(aes(x=Sepal.Width, fill=Species), color="white",  
    bins=10, position="stack")
```



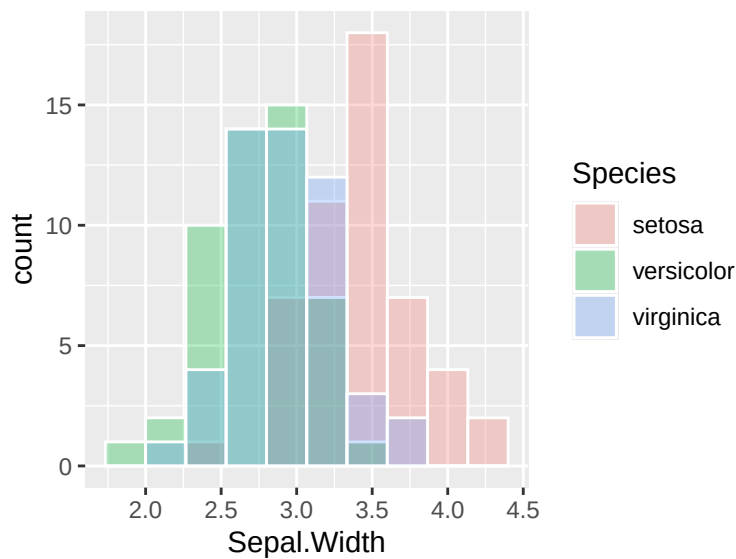
グループ設定ありの横並びヒストグラム

```
ggplot(data=iris) +
  geom_histogram(aes(x=Sepal.Width,fill=Species), color="white",
    bins=10, position="dodge")
```



グループ設定ありの重ね合わせヒストグラム（重なりがわかるよう透明度を 0.3 に設定）

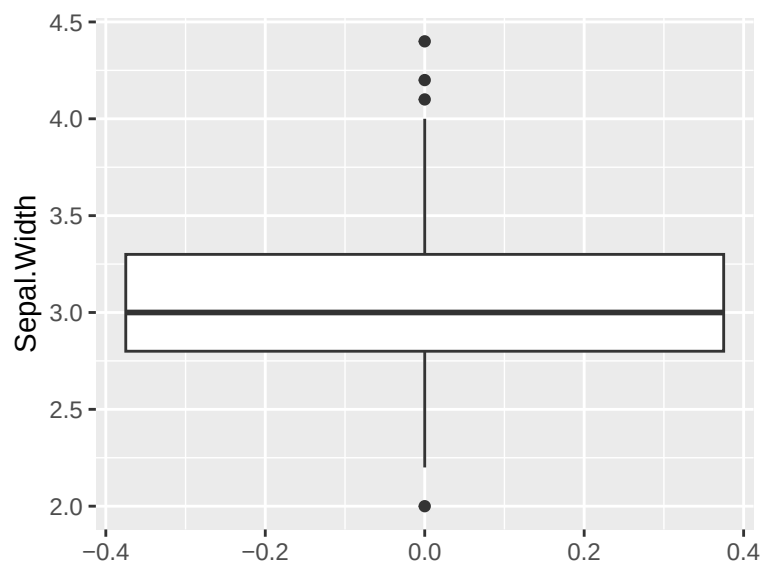
```
ggplot(data=iris) +
  geom_histogram(aes(x=Sepal.Width,fill=Species), color="white",
    bins=10, position="identity", alpha=0.3 )
```



13.4 箱ひげ図を描く

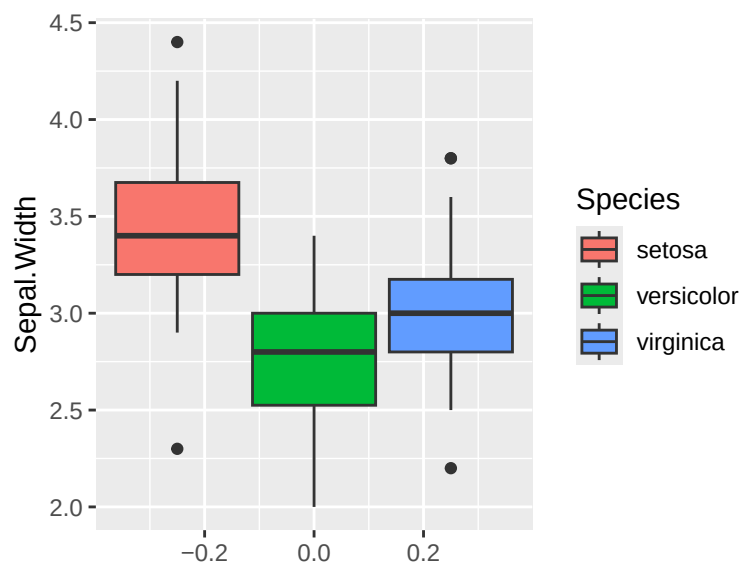
- 例文: `geom_boxplot(data, aes(x=group, y=y, fill=group), alpha=0.5)`
- 引数:
 - 第 1 引数 `data` (省略可): キャンバスに結びつけるデータフレームオブジェクト. 先に `ggplot()` 内で指定している場合は省略可能.
 - 第 2 引数 `aes()` (省略可): 先に `ggplot()` 内で指定している場合は省略可能. 留意点は以下の通り.
 - * 第 1 引数 `y`: 箱ひげ図を描く対象の変数を指定する.
 - * 第 2 引数 `x` と `fill` (省略可): 箱ひげ図を描くためのグループ分けを行う変数を指定する. 省略した場合, 全てのデータが一つにまとめられた箱ひげ図が描かれる. `x` を指定すると箱ひげ図の横軸がグループ名になる. `fill` を指定するとグループごとに箱ひげ図が塗分けられる.
 - 第 3 引数 `alpha=0.5` (省略可): 箱ひげ図の透明度を指定する.
- 返回值: 箱ひげ図が描かれた `ggplot` オブジェクト
- 留意点: `x` と `y` を入れ替えると, 箱ひげ図の向きが変わる. 例えばワイド型に並んだ 3 つの変数の箱ひげ図をまとめて描画させたい場合には, ロング型データに変換して, それら 3 つの変数を一つの変数にまとめておく必要がある.

```
# 通常の箱ひげ図 (グループ設定なし)
ggplot(data=iris) +
  geom_boxplot(aes(y=Sepal.Width))
```

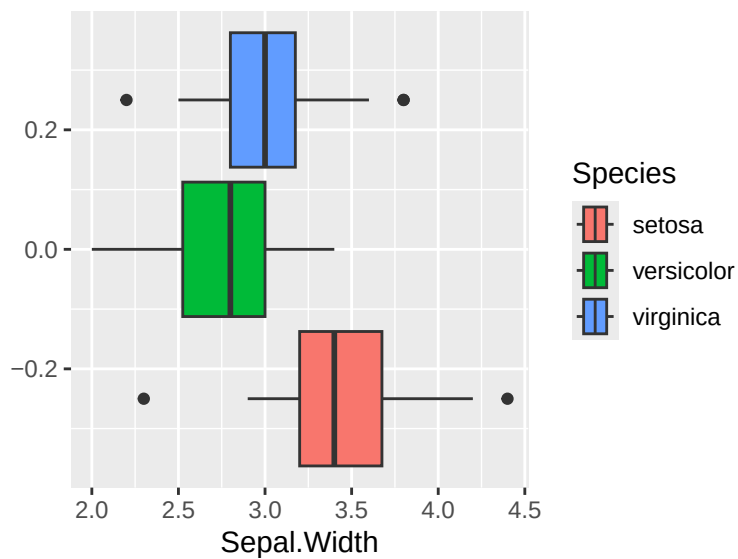
グループ設定ありの箱ひげ図

```
ggplot(data=iris) +  
  geom_boxplot(aes(y=Sepal.Width, fill=Species))
```



グループ設定ありの箱ひげ図

```
ggplot(data=iris) +  
  geom_boxplot(aes(x=Sepal.Width, fill=Species))
```

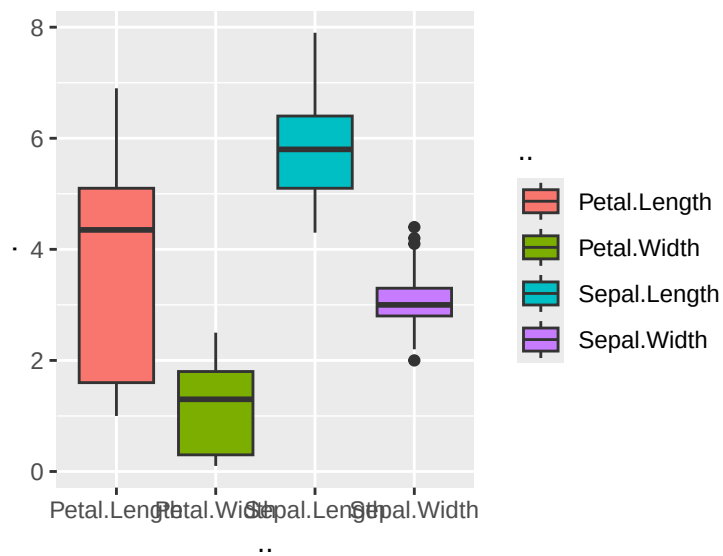


```
# 4つの変数の箱ひげ図
## "Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width" が
## ワイド型に並んでいるのを確認
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1          3.5          1.4          0.2  setosa
## 2          4.9          3.0          1.4          0.2  setosa
## 3          4.7          3.2          1.3          0.2  setosa
## 4          4.6          3.1          1.5          0.2  setosa
## 5          5.0          3.6          1.4          0.2  setosa
## 6          5.4          3.9          1.7          0.4  setosa
```

```
## "Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width" の4つの変数を
## ロング型に変換する
iris_long <- pivot_longer(iris,
                           cols = c(Sepal.Length, Sepal.Width,
                                     Petal.Length, Petal.Width),
                           names_to = "変数",
                           values_to = "値")

## 箱ひげ図を描画
ggplot(data=iris_long) +
  geom_boxplot(aes(y=値, x=変数, fill=変数))
```



13.5 グループごとの平均の棒グラフとエラーバーを描く

データそのものを与える場合と、予め平均や標準偏差を算出して与える場合とで書き方が変わる。

13.5.1 データそのものを与える場合

- 例文:

```
geom_bar(data, aes(x=group, y=y, fill=group), stat="summary", fun="mean",
width=0.5)
```

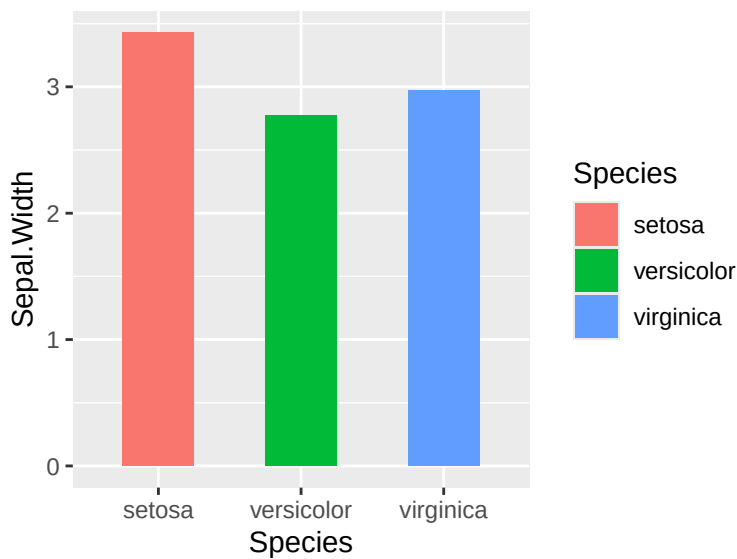
- 引数:

- 第 1 引数 `data` (省略可): キャンバスに結びつけるデータフレームオブジェクト。先に `ggplot()` 内で指定している場合は省略可能。
- 第 2 引数 `aes()` (省略可): 先に `ggplot()` 内で指定している場合は省略可能。
- 第 3 引数 `stat="summary"`: 棒グラフの高さをどのように計算するかを指定する。“summary”を指定したばあい、`fun` で指定した関数を用いて、`y` の平均値を自動で計算する。
- 第 4 引数 `fun="mean"` (省略可): `stat=summary` のときにその計算方法を指定する。“mean”で平均値を計算する。他には“median”（中央値）や“max”（最大値）などが指定できる。
- 第 5 引数 `width=0.5` (省略可): 棒グラフの幅を指定する。

- 返回值: 平均の棒グラフが描かれた `ggplot` オブジェクト

`data` と `aes` は `ggplot()` で与えた。

```
ggplot(data=iris, aes(x=Species, y=Sepal.Width, fill = Species)) +
  geom_bar(stat="summary", fun="mean", width=0.5)
```



■13.5.1.1 エラーバーの追記

- 例文:

```
geom_errorbar(data, aes(x=group, y=y), stat = "summary", fun.data = mean_se,
width = 0.2)
```

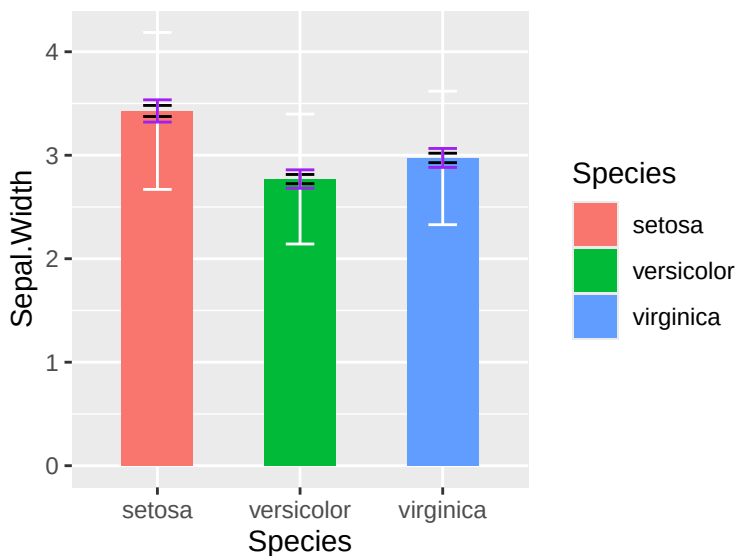
- 引数:

- 第1引数 `data` (省略可): キャンバスに結びつけるデータフレームオブジェクト。先に `ggplot()` 内で指定している場合は省略可能。
- 第2引数 `aes()` (省略可): 先に `ggplot()` 内で指定している場合は省略可能。
- 第3引数 `stat="summary"`: エラーバーの高さをどのように計算するかを指定する。データそのものを与える場合には "summary" を指定する。"summary" を指定した場合、次の `fun.data` で指定した関数を用いて、エラーバーの高さが自動計算される。
- 第4引数 `fun.data=mean_se`: この設定に合わせてエラーバーに描かれるものが設定される。
 - * `mean_se` にするとエラーバーが標準誤差となる。
 - * `mean_sdl` にするとエラーバーが標準偏差となる。
 - ・ `fun.data=function(x) mean_sdl(x, mult=2)` とすると、mult で標準偏差の何倍をエラーバーとするかを指定できる。
 - * `mean_cl_normal` にするとエラーバーが95%信頼区間となる。
- 第5引数 `width=0.2` (省略可): エラーバーの幅を指定する。省略すると0.9が設定される。
- 第6引数 `color` (省略可): エラーバーの色を指定する。省略すると "black" になる。

```
# data と aes は ggplot() で与える
## 黒 (デフォルト): 標準誤差
## 白: 標準偏差
```

紫: 95% 信頼区間

```
ggplot(data=iris, aes(x=Species, y=Sepal.Width, fill = Species)) +
  geom_bar(stat="summary", fun="mean",width=0.5) +
  geom_errorbar(stat = "summary", fun.data = mean_se, width=0.2)+
  geom_errorbar(stat = "summary", fun.data = function(x) mean_sdl(x, mult=2),
    width = 0.2, color = "white")+
  geom_errorbar(stat = "summary", fun.data = mean_cl_normal,
    width = 0.2, color = "purple")
```



13.5.2 平均や標準偏差を算出して与える場合

あらかじめ平均値や標準偏差（標準誤差や95%信頼区間でも良い）を求め、次のようなデータフレームとして保存しておく。

```
data.frame(group=各グループ名のベクトル,
  M=各グループの平均値のベクトル,
  E=各グループの標準偏差のベクトル)
```

- 例文:

```
geom_bar(data, aes(x=group, y=M, fill=group), stat="identity", width=0.5)
```

- 引数:

- 第1引数 `data` (省略可): キャンバスに結びつけるデータフレームオブジェクト. 先に `ggplot()` 内で指定している場合は省略可能.
- 第2引数 `aes()` (省略可): 先に `ggplot()` 内で指定している場合は省略可能.
 - * `x`: グループ変数を指定する.

- * `y`: 平均値を収めた変数を指定する.
- * `fill` (省略可): グループごとに色分けする変数を指定する.
- 第 3 引数 `stat="identity"`: 棒グラフの高さをどのように計算するかを指定する. “identity” を指定した場合, `y` の値がそのまま棒グラフの高さになる.
- 第 4 引数 `width=0.5` (省略可): 棒グラフの幅を指定する.
- 返回值: 平均の棒グラフが描かれた `ggplot` オブジェクト

■13.5.2.1 エラーバーの追記

- 例文:


```
geom_errorbar(data, aes(ymin=M-E, ymax=M+E),
               width = 0.2)
```
- 引数:
 - 第 1 引数 (省略可): キャンバスに結びつけるデータフレームオブジェクト. 先に `ggplot()` 内で指定している場合は省略可能.
 - 第 2 引数 `aes()`: `ymin` と `ymax` でエラーバーの下限と上限を指定する. ここで `M` と `E` を使う.
 - 第 3 引数 `width=0.2` (省略可): エラーバーの幅を指定する.
 - 第 4 引数 `color` (省略可): エラーバーの色を指定する. 省略すると “black” になる.

平均値と標準偏差を算出

```
M <- aggregate(Sepal.Width~Species, data=iris, mean)
```

```
E <- aggregate(Sepal.Width~Species, data=iris, sd)
```

グループごとの平均値と標準偏差をデータフレーム化

```
df_m_sd <- data.frame(Species=M$Species, M=M$Sepal.Width, E=E$Sepal.Width)
```

一旦出力

```
df_m_sd
```

```
##      Species      M      E
## 1    setosa 3.428 0.3790644
## 2 versicolor 2.770 0.3137983
## 3  virginica 2.974 0.3224966
```

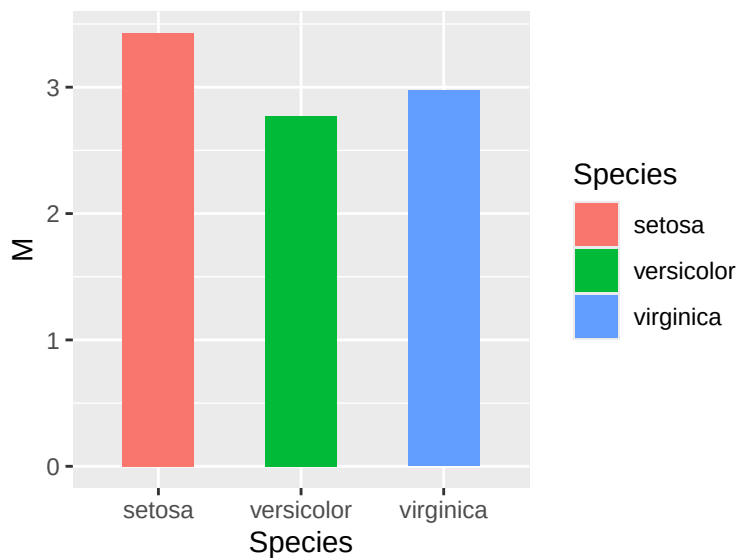
平均値と標準偏差を与える

以下では `data` と `aes` は `ggplot()` で与えている

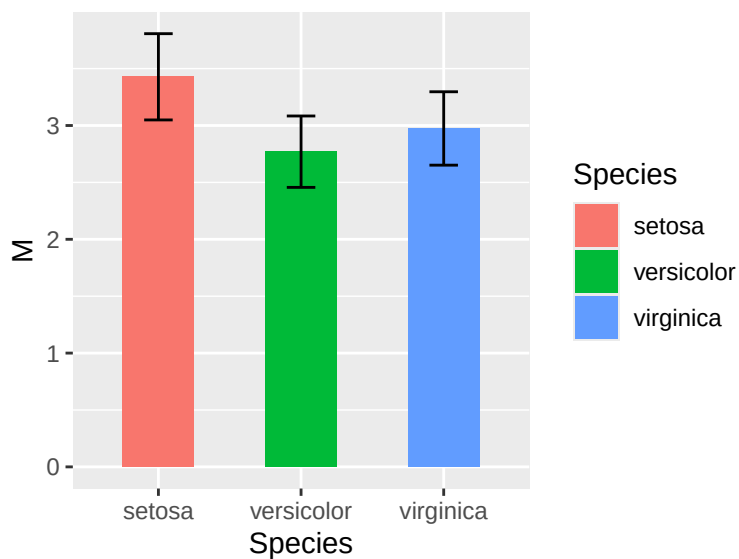
`ggplot` オブジェクトに保存したのち, `plot` で出力させている

```
g<-ggplot(df_m_sd, aes(x=Species, y=M, fill = Species)) +
```

```
geom_bar(stat="identity", width=0.5)
plot(g)
```



```
# 先の ggplot オブジェクトにエラーバーを追記
g<-g+geom_errorbar(aes(ymin=M-E, ymax=M+E), width = 0.2)
plot(g)
```



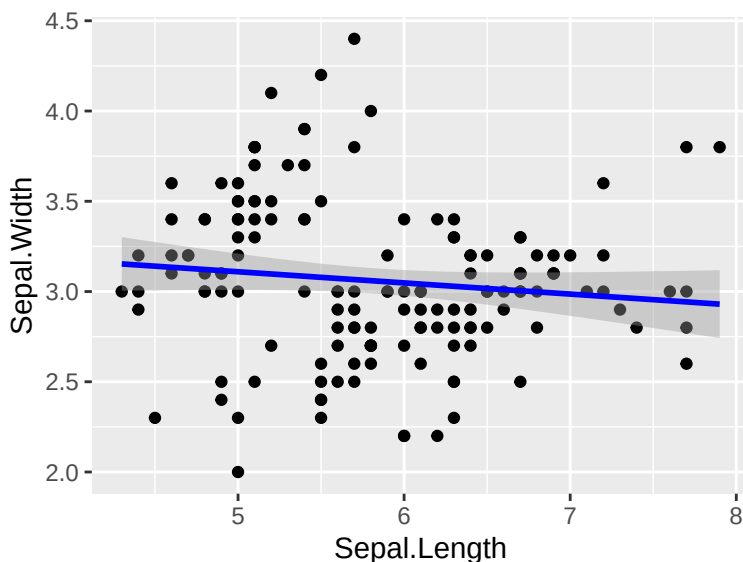
13.6 回帰直線と信頼区間を描く

- 例文: `geom_smooth(data, aes(x=x, y=y, color=group), method="lm", se=TRUE)`

- 引数:
 - 第1引数 `data` (省略可): キャンバスに結びつけるデータフレームオブジェクト。先に `ggplot()` 内で指定している場合は省略可能。
 - 第2引数 `aes()` (省略可): 先に `ggplot()` 内で指定している場合は省略可能。
 - * 第1引数 `x`: x 軸に対応する変数を指定する。
 - * 第2引数 `y`: y 軸に対応する変数を指定する。
 - * 第3引数 `color` (省略可): グループごとに回帰直線の描く場合には、グループ変数を指定する。省略した場合、全てのデータを一つにまとめた回帰直線が描かれる。
 - 第3引数 `method="lm"`: 回帰直線の計算方法を指定する。“lm”を指定すると最小二乗法による回帰直線が描かれる。
 - 第4引数 `se=TRUE` (省略可): `TRUE` にすると信頼区間が描かれる。
- 返り値: 回帰直線と信頼区間が描かれた `ggplot` オブジェクト
- 留意点: 一般に回帰直線と信頼区間を描くときには散布図に重ね合わせることが多い。その場合、`geom_point()` を用いて散布図を描いた後に `geom_smooth()` を用いて回帰直線を描く。

```
# data と aes は ggplot() で与えた。color は設定せず
ggplot(data=iris, aes(x=Sepal.Length, y=Sepal.Width)) +
  geom_point() +
  geom_smooth(method="lm", se=TRUE, color="blue")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

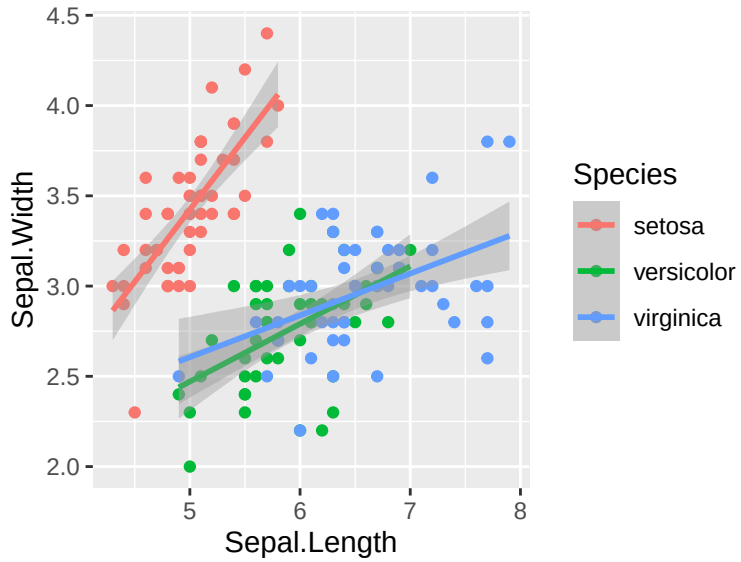


```
# data と aes は ggplot() で与えた。color も設定してグループ別に回帰直線を引いた
ggplot(data=iris, aes(x=Sepal.Length, y=Sepal.Width, color = Species)) +
  geom_point() +
```



```
geom_smooth(method="lm", se=TRUE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



13.7 任意の直線を描く